

ITSS GROUP 25

1. Design for class “Product”

<<entity>> Product
- id : string - title : string - category : string - barcode : string - originalValue : number - currentPrice : number - height : number - width : number - length : number - weight : number - generalDescription : string - quantityInStock : number - status : ProductStatus
+ Product(id : string, title : string, category : string, barcode : string, originalValue : number, currentPrice : number, weight : number, height : number, width : number, length : number, generalDescription : string, quantityInStock : number, status : ProductStatus) + getId() : string + getTitle() : string + getCategory() : string + getBarcode() : string + getOriginalValue() : number + getCurrentPrice() : number + getWeight() : number + getHeight() : number + getWidth() : number + getLength() : number + getGeneralDescription() : string + getQuantityInStock() : number + getStatus() : ProductStatus + setTitle(title : string) : void + setCategory(category : string) : void + setBarcode(barcode : string) : void + setOriginalValue(originalValue : number) : void + isValidPrice(newPrice : number) : boolean + setWeight(weight : number) : void + setHeight(height : number) : void + setWidth(width : number) : void + setLength(length : number) : void + setGeneralDescription(generalDescription : string) : void + updateCurrentPrice(newPrice : number) : void + adjustStock(detail : number, reason : string) : void + isAvailableForSale() : boolean + canBeDeleted() : boolean + deactivate() : void

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	id	string	generated by system	Unique identifier of the product
2	title	string	""	Name of the product
3	category	string	""	Category of the product
4	barcode	string	""	Barcode of the product
5	originalValue	number	0	Original value / import value of the product
6	currentPrice	number	0	Current selling price of the product
7	height	number	0	Height of the product
8	width	number	0	Width of the product
9	length	number	0	Length of the product

10	weight	number	0	Weight of the product
11	generalDescription	string	""	General description of the product
12	quantityInStock	number	0	Quantity of product currently in stock
13	status	ProductStatus	Active	Current status of the product

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	Product(id, title, category, barcode, originalValue, currentPrice, weight, height, width, length, generalDescription, quantityInStock, status)	constructor	Create a new Product object with full information
2	getId()	string	Return the product id
3	getTitle()	string	Return the product title
4	getCategory()	string	Return the product category

5	getBarcode()	string	Return the product barcode
6	getOriginalValue()	number	Return the original value
7	getCurrentPrice()	number	Return the current price
8	getWeight()	number	Return the product weight
9	getHeight()	number	Return the product height
10	getWidth()	number	Return the product width
11	getLength()	number	Return the product length
12	getGeneralDescription()	string	Return the general description
13	getQuantityInStock()	number	Return the stock quantity
14	getStatus()	ProductStatus	Return the current status
15	setTitle(title)	void	Update the title of the product
16	setCategory(category)	void	Update the category of the product
17	setBarcode(barcode)	void	Update the barcode of the product

18	setOriginalValue(original Value)	void	Update the original value
19	setCurrentPrice(currentPrice)	boolean	Update the current selling price
20	setWeight(weight)	void	Update the weight of the product
21	setHeight(height)	void	Update the height of the product
22	setWidth(width)	void	Update the width of the product
23	setLength(length)	void	Update the length of the product
24	setGeneralDescription(generalDescription)	void	Update the general description
25	isValidPriceForSale()	boolean	Check whether the current price is valid for selling
26	changeQuantityInStock(detail)	void	Adjust stock quantity based on detail and reason
27	isAvailableForSale()	boolean	Check whether the product is available for sale
28	canBeDeleted()	boolean	Check whether the product can be deleted

29	changeQuantityInStock(delta)	void	Change stock quantity by a positive or negative amount
30	setStatus(status)	void	Update the product status

Detailed specification of main operations

1. Product(id, title, category, barcode, originalValue, currentPrice, weight, height, width, length, generalDescription, quantityInStock, status)

Parameter:

- id: unique identifier of the product
- title: name of the product
- category: category of the product
- barcode: barcode of the product
- originalValue: original value of the product
- currentPrice: current selling price of the product
- weight: weight of the product
- height: height of the product
- width: width of the product
- length: length of the product
- generalDescription: general description of the product
- quantityInStock: quantity currently in stock
- status: current status of the product

Exception:

- IllegalArgumentException if title, category, or barcode is empty
- IllegalArgumentException if originalValue, currentPrice, weight, height, width, length, or quantityInStock is less than 0
- IllegalArgumentException if status is null or invalid

Method

- Initialize a Product object with full product information
- Validate all input values before assigning them to attributes
- Set the initial status of the product to a valid state
- Ensure the product object is created in a consistent state

Other diagrams

- State diagram if any

2. setCurrentPrice(currentPrice)

Parameter:

- currentPrice: new selling price of the product

Exception:

- IllegalArgumentException if currentPrice < 0

Method

- Validate the new price before updating
- If the price is valid, assign it to currentPrice
- This operation is allowed when the product is in Active or Deactivated state
- The method does not change the product state

Other diagrams

- State diagram / activity diagram if needed

3. changeQuantityInStock(delta)

Parameter:

- delta: stock quantity to be added or removed
 - positive value means increasing stock
 - negative value means decreasing stock

Exception:

- IllegalArgumentException if the resulting stock quantity is less than 0

Method

- Update stock quantity using the formula:
 $\text{quantityInStock} = \text{quantityInStock} + \text{delta}$
- Ensure stock quantity is never negative
- This operation is allowed when the product is in Active or Deactivated state
- The method does not directly change the product state, but the updated quantity may affect deletion eligibility

Other diagram

- State diagram / activity diagram if needed

4. isValidPriceForSale()

Parameter:

- None

Exception:

- None

Method

- Check whether the current price is valid for sale
- Return true if `currentPrice >= 0`
- Return false otherwise

Other diagram

- Activity diagram if the validation logic is considered complex

5. isAvailableForSale()

Parameter:

- None

Exception:

- None

Method

- Check whether the product is available for sale
- Return true if:
 - `status = Active`
 - `quantityInStock > 0`
 - `isValidPriceForSale()` returns true
- Return false otherwise

Other diagram

- State diagram if any

6. canBeDeleted()

Parameter:

- None

Exception:

- None

Method

- Check whether the product can be deleted from the system
- Return true if quantityInStock = 0
- Return false if quantityInStock > 0
- According to the state logic:
 - if a deletion request occurs when quantityInStock = 0, the product may move to Deleted
 - if a deletion request occurs when quantityInStock > 0, the product should first move to Deactivated

Other diagram

- State diagram if any

7. deactivate()

Parameter:

- None

Exception:

- IllegalStateException if the product is already in Deleted state
- IllegalStateException if the product cannot be deactivated according to business rules

Method

- Change the product status from Active to Deactivated
- Prevent the product from being sold after deactivation
- Allow product information such as price or stock quantity to still be updated if needed
- This method is typically used when a deletion request is made but the product still has stock quantity greater than 0

Other diagram

- State diagram if any

8. Getter methods

8.1. getId()

Parameter:

- None

Exception:

- None

Method

- Return the product id

Other diagram

- None

8.2. getTitle()

Parameter:

- None

Exception:

- None

Method

- Return the product title

Other diagram

- None

8.3. getCategory()

Parameter:

- None

Exception:

- None

Method

- Return the product category

Other diagram

- None

8.4. getBarcode()

Parameter:

- None

Exception:

- None

Method

- Return the product barcode

Other diagram

- None

8.5. getOriginalValue()**Parameter:**

- None

Exception:

- None

Method

- Return the original value

Other diagram

- None

8.6. getCurrentPrice()**Parameter:**

- None

Exception:

- None

Method

- Return the current selling price

Other diagram

- None

8.7. getWeight()**Parameter:**

- None

Exception:

- None

Method

- Return the product weight

Other diagram

- None

8.8. getHeight()

Parameter:

- None

Exception:

- None

Method

- Return the product height

Other diagram

- None

8.9. getWidth()

Parameter:

- None

Exception:

- None

Method

- Return the product width

Other diagram

- None

8.10. getLength()

Parameter:

- None

Exception:

- None

Method

- Return the product length

Other diagram

- None

8.11. getGeneralDescription()

Parameter:

- None

Exception:

- None

Method

- Return the general description

Other diagram

- None

8.12. getQuantityInStock()

Parameter:

- None

Exception:

- None

Method

- Return the quantity in stock

Other diagram

- None

8.13. getStatus()**Parameter:**

- None

Exception:

- None

Method

- Return the product status

Other diagram

- None

9. Setter methods**9.1. setTitle(title)****Parameter:**

- title: new title of the product

Exception:

- IllegalArgumentException if title is empty

Method

- Update the title of the product

Other diagram

- None

9.2. setCategory(category)**Parameter:**

- category: new category of the product

Exception:

- IllegalArgumentException if category is empty

Method

- Update the category of the product

Other diagram

- None

9.3. setBarcode(barcode)**Parameter:**

- barcode: new barcode of the product

Exception:

- IllegalArgumentException if barcode is empty

Method

- Update the barcode of the product

Other diagram

- None

9.4. setOriginalValue(originalValue)**Parameter:**

- originalValue: new original value

Exception:

- IllegalArgumentException if originalValue < 0

Method

- Update the original value of the product

Other diagram

- None

9.5. setCurrentPrice(currentPrice)**Parameter:**

- currentPrice: new current price

Exception:

- IllegalArgumentException if currentPrice < 0

Method

- Update the current selling price of the product

Other diagram

- None

9.6. setWeight(weight)

Parameter:

- weight: new weight of the product

Exception:

- IllegalArgumentException if weight < 0

Method

- Update the weight of the product

Other diagram

- None

9.7. setHeight(height)

Parameter:

- height: new height of the product

Exception:

- IllegalArgumentException if height < 0

Method

- Update the height of the product

Other diagram

- None

9.8. setWidth(width)

Parameter:

- width: new width of the product

Exception:

- IllegalArgumentException if width < 0

Method

- Update the width of the product

Other diagram

- None

9.8. setLength(length)

Parameter:

- length: new length of the product

Exception:

- IllegalArgumentException if length < 0

Method

- Update the length of the product

Other diagram

- None

9.10. setGeneralDescription(generalDescription)

Parameter:

- generalDescription: new general description

Exception:

- None

Method

- Update the general description of the product

Other diagram

- None

9.11. setGeneralDescription(generalDescription)

Parameter:

- status: new status of the product

Exception:

- IllegalArgumentException if status is null or invalid

Method

- Update the status of the product
- The status should be one of the supported states in the system, such as Active, Deactivated, or Deleted

Other diagram

- State diagram if any

2. Design for class “ProductListScreen”

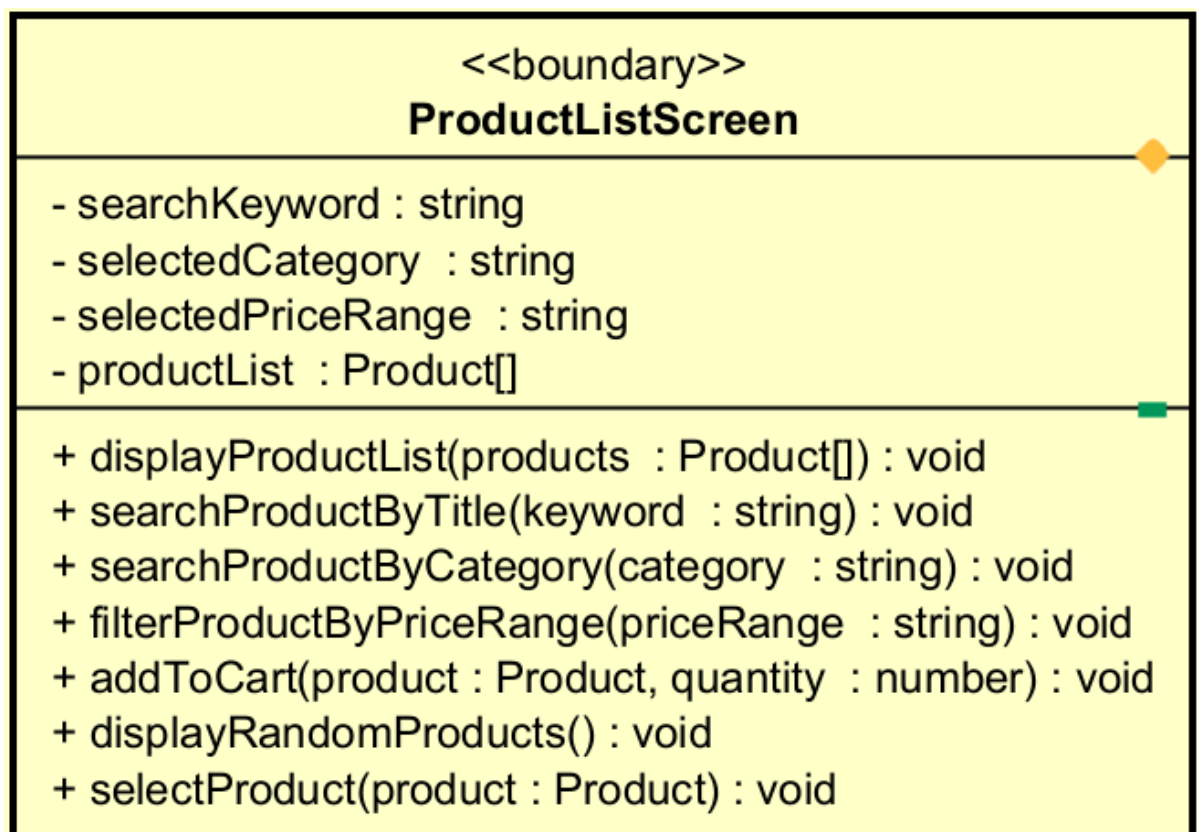


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	searchKeyword	string	""	Keyword entered by user to search products by title
2	selectedCategory	string	""	Category selected by user for filtering products
3	selectedPriceRange	string	""	Price range selected by user for filtering products
4	productList	Product[]	empty list	List of products displayed on the screen

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayProductList(products : Product[])	void	Display a list of products on the product list screen
2	searchProductByTitle(keyword : string)	void	Search products by title based on the entered keyword

3	searchProductByCategory (category : string)	void	Filter products by selected category
4	filterProductByPriceRange (priceRange : string)	void	Filter products by selected price range
5	addToCart(product : Product, quantity : number)	void	Add a selected product with quantity to cart
6	displayRandomProducts()	void	Display a random list of products for recommendation
7	selectProduct(product : Product)	void	Select a product from the list to view its detail

Detailed specification of main operations

1. displayProductList(products : Product[])

Parameter:

- products: list of products to be displayed on the screen

Exception:

- None

Method

- Receive a list of products from the system
- Assign the list to productList
- Render product items on the user interface
- Show common product information such as title, category, current price, stock status, and image if available
- The displayed list may contain objects of subclasses of Product, such as DVD, CD, Book, or Newspaper

Other diagram

- Sequence diagram if the interaction with controller/service is shown

2. searchProductByTitle(keyword : string)

Parameter:

- keyword: keyword entered by the user

Exception:

- Validation exception if keyword is null
- No exception if keyword is empty, the system may display all products

Method

- Save the entered keyword into searchKeyword
- Search products whose title matches or contains the keyword
- Update the displayed productList
- Refresh the screen with the search result

Other diagram

- Activity diagram / sequence diagram if the searching process is shown in detail

3. searchProductByCategory(category : string)

Parameter:

- category: category selected by the user

Exception:

- Validation exception if category is null or invalid

Method

- Save the selected category into selectedCategory
- Filter the products based on the selected category
- Update the displayed productList
- Refresh the screen with filtered products

Other diagram

- Activity diagram if the filtering logic is described in detail

4. filterProductByPriceRange(priceRange : string)

Parameter:

- priceRange: selected price range for filtering

Exception:

- Validation exception if priceRange is null or not supported by the system

Method

- Save the selected price range into selectedPriceRange
- Filter the products whose prices belong to the selected range
- Update the displayed productList
- Refresh the screen after filtering

Other diagram

- Activity diagram if the filtering logic is considered complex

5. addToCart(product : Product, quantity : number)**Parameter:**

- product: selected product to be added to cart
- quantity: quantity chosen by the user

Exception:

- Validation exception if product is null
- Validation exception if quantity ≤ 0
- Validation exception if quantity is greater than available stock
- Validation exception if the product is not available for sale

Method

- Check whether the selected product is valid
- Check whether the quantity is valid
- Check whether product.isAvailableForSale() returns true
- Ensure the product status is Active
- Ensure the requested quantity does not exceed quantityInStock
- Send add-to-cart request to the cart subsystem
- Update cart information if the action is successful
- Notify the user if the product is added successfully

Other diagram

- Sequence diagram is recommended because this method usually interacts with Cart / CartController / Product

6. displayRandomProducts()**Parameter:**

- None

Exception:

- None

Method

- Request a random set of products from the system
- Assign the returned list to productList
- Display recommended or random products on the screen

Other diagram

- Sequence diagram if recommendation logic is shown

7. selectProduct(product : Product)

Parameter:

- product: product selected by the user

Exception:

- Validation exception if product is null

Method

- Save the selected product temporarily
- Trigger navigation to the product detail screen
- Pass the selected product information to the next screen
- The selected product may be an instance of a subclass of Product, such as DVD, CD, Book, or Newspaper
- The detail screen will determine the actual product type and display the corresponding specific information

Other diagram

- Sequence diagram is recommended because this method usually leads to screen navigation

3. Design for class “ProductDetailScreen”

<<boundary>> ProductDetailScreen	
- selectedProduct : Product - selectedQuantity : number	
+ displayProductDetail(product : Product) : void + showTypeSpecificInfo(product : Product) : void + addToCart(product : Product, quantity : number) : void + backToProductList() : void	

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	selectedProduct	Product	null	Product currently selected to display in detail
2	selectedQuantity	number	1	Quantity selected by the user before adding to cart

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayProductDetail(product : Product)	void	Display full detail information of the selected product

2	showTypeSpecificInfo(product : Product)	void	Display additional information depending on product type or category
3	addToCart(product : Product, quantity : number)	void	Add the selected product with the selected quantity to cart
4	backToProductList()	void	Navigate back to the product list screen

Detailed specification of main operations

1. displayProductDetail(product : Product)

Parameter:

- product: product whose details are displayed

Exception:

- Validation exception if product is null

Method

- Assign the given product to selectedProduct
- Display common information of the product on the screen
- Show information such as title, category, barcode, current price, dimensions, weight, stock quantity, general description, and status
- Call showTypeSpecificInfo(product) to display additional information based on the actual product type
- Update the interface accordingly

Other diagram

- Sequence diagram if the detail loading process is shown

2. showTypeSpecificInfo(product : Product)

Parameter:

- product: selected product

Exception:

- Validation exception if product is null

Method

- Determine the actual type of the selected product
- Display additional information specific to that type
- If the product is a DiscProduct, display common disc information such as genre and releaseDate
- If the product is a PrintableProduct, display common printable information such as publisher, publicationDate, and language
- If the product is a DVD, display director, runtime, studio, and subtitles
- If the product is a CD, display artists, recordLabel, and track list if available
- If the product is a Book, display author, coverType, and numberOfPages
- If the product is a Newspaper, display editorInChief, issueNumber, publicationFrequency, issn, and sections

Other diagram

- Activity diagram if the branching logic by product type is shown

3. addToCart(product : Product, quantity : number)**Parameter:**

- product: selected product to add to cart
- quantity: quantity selected by the user

Exception:

- Validation exception if product is null
- Validation exception if quantity ≤ 0
- Validation exception if quantity is greater than available stock
- Validation exception if the product is not available for sale

Method

- Check whether the product is valid
- Check whether the entered quantity is valid
- Check whether product.isAvailableForSale() returns true
- Ensure the product status is Active
- Ensure the requested quantity does not exceed quantityInStock
- Add the selected product to cart
- Update cart information after the action is completed
- Notify the user about the result

Other diagram

- Sequence diagram is recommended because this method interacts with cart-related classes

4. backToProductList()

Parameter:

- None

Exception:

- None

Method

- Close or leave the current product detail screen
- Navigate back to the product list screen
- Keep or restore previous search/filter result if required by the system

Other diagram

- Sequence diagram if navigation flow is shown

4. Design for class “ProductEditPage”

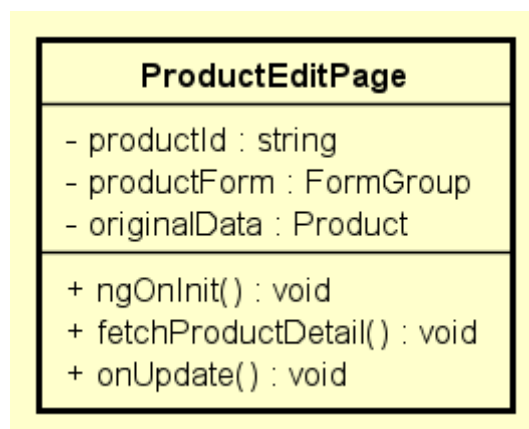


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	string	null	Stores the unique identifier of the product currently being edited.
2	productForm	FormGroup	null	An Angular Reactive Form object used to manage and validate user input from the UI.
3	originalData	Product	null	Holds the initial data of the product to allow for data comparison or form resetting.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	ngOnInit	void	Initializes the component by extracting the productId from the route and triggering the data fetch.
2	fetchProductDetail	void	Calls the ProductApiService to retrieve the current details of the product to be edited.
3	onUpdate	void	Collects data from the form and sends a request to the system to update the product information.

Parameter:

- productId (in ngOnInit): string - The ID extracted from the route parameters to identify which product to modify.
- data (result from fetchProductDetail): ProductDetailResponseDto - The data object returned from the server used to populate the form fields.

Exception:

- **ProductNotFoundException:** Triggered and displayed on the UI if the provided product ID does not exist in the system.
- **InvalidPriceException:** Triggered if the manager enters a price that does not comply with the 30% - 150% original value rule.

Method:

- **fetchProductDetail():**
 1. Invoke `getProductById(productId)` from `ProductApiService`.
 2. Upon success, assign the result to `originalData` and use `patchValue` to populate the `productForm`.
- **onUpdate():**
 1. Perform form validation (Angular Validators).
 2. If valid, encapsulate the data into an `UpdateProductDto` (ensuring the barcode field is excluded).
 3. Call `ProductApiService.updateProduct()` and navigate back to the product list upon a successful response.

5. Design for class "ProductCreatePage"

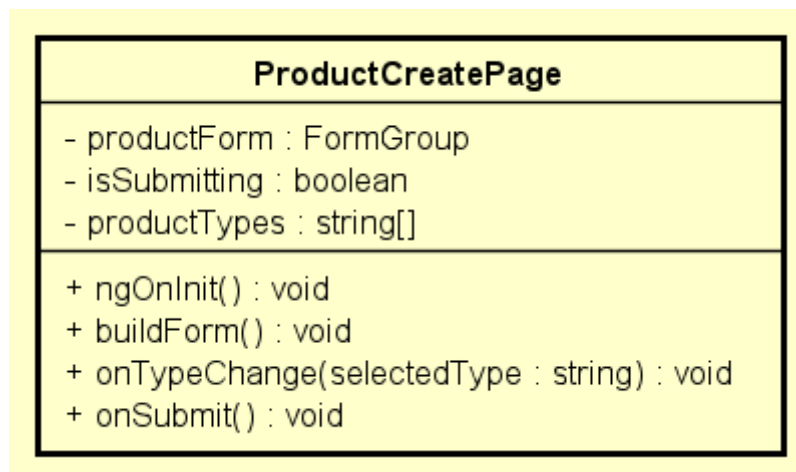


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productForm	FormGroup	null	Manages the form state, validation, and user input data.

2	isSubmitting	boolean	false	A flag used to disable the submit button and show a loading state during API calls.
3	productTypes	string[]	['Book', 'CD', 'DVD', 'Newspaper']	A static list of available product types to populate the selection menu.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	ngOnInit	void	Lifecycle hook that triggers initialization logic when the page loads.
2	buildForm	void	Initializes the productForm with default values and validation rules (e.g., required fields).
3	onTypeChange	void	Updates the form structure dynamically based on the selected product type.
4	onSubmit	void	Collects form data and triggers the product creation process.

Parameter:

- selectedType: string — The value chosen from the dropdown (e.g., "Book", "CD") to determine which extra fields to display.

Exception:

- ValidationError: Triggered if the user attempts to submit the form while mandatory fields (like barcode or originalPrice) are empty or invalid.

Method Design (Logic/Algorithm):

1. buildForm()

1. Initialize productForm with common fields: title, category, barcode, originalPrice, weight, and dimensions.
2. Set validation constraints: barcode must be unique, and originalPrice must be $\$ > 0\$$.

2. onTypeChange(selectedType)

1. Detect the change event from the product type dropdown.
2. Logic: If "Book" is selected, dynamically add author and publisher fields to the form. If "CD" is selected, add artist and tracklist.
3. Ensure that the ProductDetailResponseDto structure will match these dynamic fields for future viewing.

3. onSubmit()

1. Set isSubmitting to true.
2. Verify form validity. If invalid, highlight errors and stop.
3. Data Packaging: Map the values from productForm into a CreateProductDto object.
4. Invoke ProductApiService.createProduct(payload).
5. Upon success, navigate the user back to the ProductListPage and reset isSubmitting.

6. Design for class "ProductListPage"

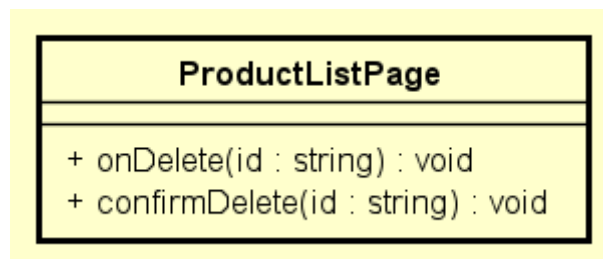


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	None	N/A	N/A	This class does not define any internal attributes in the current UML design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	onDelete	void	Triggers when the manager clicks the delete button for a specific product or a group of products.
2	confirmDelete	void	Displays a confirmation dialog to the user before proceeding with the actual deletion request.

Parameter:

- id (in onDelete/confirmDelete): string - The unique identifier of the product targeted for deletion.
- ids (optional for bulk): string[] - A collection of up to 10 product IDs selected for simultaneous deletion.

Exception:

- DeletionConstraintException: Displayed if the user attempts to delete more than 10 products at once or exceeds the daily limit of 20.
- ConnectionError: Displayed if the UI fails to communicate with the ProductApiService.

Method:

- onDelete(id):
 1. Capture the id of the product(s) the user wants to remove.
 2. Store the id temporarily and invoke confirmDelete() to prevent accidental data loss.
- confirmDelete(id):
 1. Open a modal or confirmation dialog asking the user: "Are you sure you want to delete this product?".
 2. If the user confirms:
 - Call deleteProduct(ids) via ProductApiService.
 - Upon successful response, refresh the productList to reflect the changes on the UI.
 3. If the user cancels, close the dialog and take no further action.

7. Design for class "ProductDetailPage"

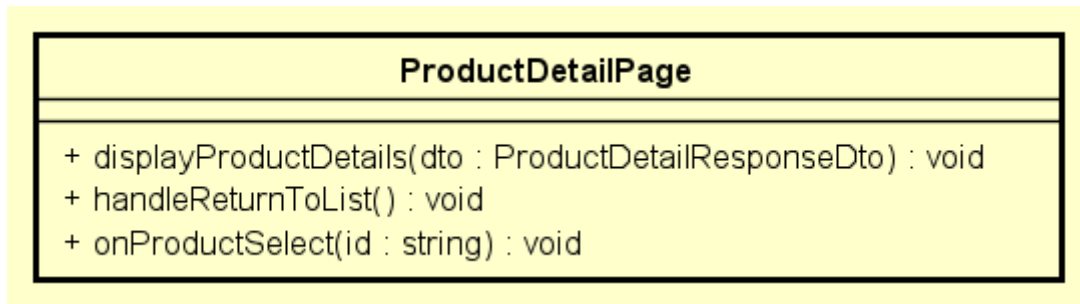


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	None	N/A	N/A	This class does not contain internal attributes as shown in the UML design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	displayProductDetails	void	Receives the fetched data and renders the specific details on the screen based on the product category.
2	handleReturnToList	void	Handles the navigation event to return the user to the main product list.
3	onProductSelect	void	Triggered when a product is selected; captures the ID and initiates the data fetch process.

Parameter:

- `dto` (in `displayProductDetails`): `ProductDetailResponseDto` - The comprehensive data object containing all general and category-specific attributes (e.g., tracklist, author, publisher).
- `id` (in `onProductSelect`): `string` - The unique identifier of the product chosen by the user.

Exception:

- `ProductNotFoundException`: Displayed as an error message or "404 Not Found" screen if the requested product ID does not exist or has been removed from the system.
- `ConnectionError`: Triggered if the UI fails to fetch data from the `ProductApiService`.

Method:

- `onProductSelect(id)`:
 1. Capture the `id` from the user's click event or the URL route.
 2. Call `ProductApiService.getProductById(id)`.
 3. Upon a successful response, pass the data to `displayProductDetails(dto)`.
- `displayProductDetails(dto)`:
 1. Assign the `dto` to the local `productDetailData` attribute.
 2. Read the category field from the DTO.
 3. Dynamically render the UI sections based on the category (e.g., show the "Author" and "Pages" layout for Books, or the "Tracklist" and "Record Label" layout for CDs).
- `handleReturnToList()`:
 1. Clear the current `productDetailData` to free up memory.
 2. Use the frontend router (e.g., Angular Router) to navigate back to the `ProductListPage`.

8. Design for class "ProductApiService"

ProductApiService
<pre>+ updateProduct(id : string, dto : UpdateProductDto) : Observable<Product> + getProductById(id : string) : Observable<ProductDetailResponseDto> + createProduct(payload : CreateProductDto) : Observable<Product> + deleteProduct(id : string) : Observable<void></pre>

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	None	N/A	N/A	This class is a stateless service used for handling HTTP communications.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	Observable<Product>	Sends an asynchronous request to the server to update existing product data.
2	getProductById	Observable<ProductDetailResponseDto>	Retrieves the complete detailed information for a specific product.
3	createProduct	Observable<Product>	Sends a request to initialize and store a new product in the database.
4	deleteProduct	Observable<void>	Requests the removal or deactivation of a product based on its stock status.

Parameter:

- id: string - The unique identifier used by the back-end to locate the specific product.
- dto: UpdateProductDto - The data object containing only the fields allowed for modification (excluding barcode).

- payload: CreateProductDto - The complete data object required to create a new product entry.

Exception:

- **HttpErrorResponse:** Handled if the network connection fails or if the server returns an error status (e.g., 404 Not Found, 500 Internal Error).
- **UnauthorizedError:** Occurs if a user tries to access protected management functions without valid credentials.

Method:

- **getProductById(id):**
 1. Executes a GET request to the backend endpoint using the product id.
 2. Maps the resulting data to the ProductDetailResponseDto structure to support the full display requirements (e.g., Title, Category, Authors).
- **deleteProduct(id):**
 1. Issues a DELETE request to the API.
 2. If the product has remaining stock, the backend logic (invoked via this API call) will transition the product to a "deactivated" state rather than performing a physical deletion.
- **updateProduct(id, dto):**
 1. Sends a PUT/PATCH request containing the dto.
 2. This method specifically avoids sending the barcode to comply with business rules preventing identification changes.

9. Design for class "ProductController"

(Insert the screenshot of your ProductController box from Astah here)

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	None	N/A	N/A	This class is a stateless controller and does not maintain internal attributes.

Table 2. Operation design

#	Name	Return type	Description (purpose)

1	update	Promise<Product>	Coordinates the update process by passing request data to the service layer.
2	create	Promise<Product>	Facilitates the creation of a new product by handling the incoming data payload.
3	remove	Promise<void>	Manages the deletion or deactivation of products for a list of provided IDs.
4	getDetail	Promise<ProductDetailResponseDto>	Requests comprehensive product information to be returned to the client.

Parameter:

- id: string — The unique identifier for a single product used in update and getDetail requests.
- dto: UpdateProductDto — The data transfer object containing the modified attributes for an existing product.
- dto (in create): CreateProductDto — The object containing all necessary information to initialize a new product.
- id (in remove): string[] — An array of product IDs selected for bulk removal or deactivation.

Exception:

- ProductNotFoundException: Rethrown if the underlying service cannot find the product for the requested ID.
- InvalidPriceException: Rethrown if the service layer detects a price update that violates business rules.
- DeletionConstraintException: Rethrown if the requested deletion batch exceeds daily limits or includes items with active stock.

Method:

- update(id, dto):

1. Receives the HTTP request and extracts the id and dto.
 2. Invokes ProductService.handleUpdate(id, dto) to perform business logic and database updates.
 3. Returns the updated Product object.
- create(dto):
 1. Validates the incoming CreateProductDto.
 2. Delegates the creation task to ProductService.createProduct(dto).
 - remove(id):
 1. Receives an array of up to 10 product IDs.
 2. Calls ProductService.handleDeletion(id) to determine whether each product should be deleted or deactivated based on stock.
 - getDetail(id):
 1. Triggers ProductService.getProductDetail(id).
 2. Ensures the returned data includes the full suite of attributes required by the "View Product Detail" specification, such as author or tracklist information.

10. Design for class "ProductService"

ProductService
<pre> + handleUpdate(id : string, dto : UpdateProductDto) : Promise<Product> + handleDeletion(id : string[]) : Promise<void> + createProduct(dto : CreateProductDto) : Promise<Product> + getProductDetail(id : string) : Promise<ProductDetailResponseDto> - checkDeletionConstraints(id : string, stockQuantity : number) : boolean - findProductOrThrow(id : string) : Promise<Product> - validateUpdateData(originalPrice : number, currentPrice : number, dto : UpdateProductDto) : boolean - trackChanges(oldData : Product, newData : UpdateProductDto) : string - validateStockQuantity() : boolean - saveHistoryLog(productId : string, action : string, reason : string) : void - deactivateProduct(id : string) : Promise<void> - ensureProductExists(id : string) : void </pre>

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	None	N/A	N/A	This service is stateless and interacts with repositories for data persistence.

Table 2. Operation design

#	Name	Return type	Description
1	handleUpdate	Promise<Product>	Main entry point for updating product information.
2	handleDeletion	Promise<void>	Main entry point for bulk product deletion or deactivation.
3	createProduct	Promise<Product>	Validates and creates a new product record.
4	getProductDetail	Promise<ProductDetailResponseDto>	Retrieves detailed info for specific product types (Book, CD, etc.).
5	checkDeletionConstraints	boolean	(Private) Validates stock and business rules for deletion.

6	findProductOrThrow	Promise<Product>	(Private) Retrieves an entity or throws an error if not found.
7	validateUpdateData	boolean	(Private) Checks the 30% - 150% price modification rule.
8	trackChanges	string	(Private) Logs the specific fields modified during an update.
9	validateStockQuantity	boolean	(Private) Validates the daily limit for product removals.
10	saveHistoryLog	void	(Private) Records actions in the system's history log.
11	deactivateProduct	Promise<void>	(Private) Sets product status to 'deactivated'

			instead of deleting.
12	ensureProductExists	void	(Private) Pre-verification for product existence.

Parameters:

- id / ids: string | string[] — Unique identifier(s) for the product(s).
- dto: UpdateProductDto | CreateProductDto — Data objects received from the Controller.
- originalPrice / currentPrice: number — Financial values used for constraint validation.
- stockQuantity: number — The current number of items in the inventory.

Exceptions:

- ProductNotFoundException: Triggered if the id does not match any record.
- InvalidPriceException: Triggered if the updated price is less than 30% or more than 150% of the original value.
- DeletionConstraintException: Triggered if the deletion exceeds 20 items per day or violates stock rules.

Method Design (Logic/Algorithm):

1. Product Detail Retrieval (getProductDetail)

1. Call ensureProductExists(id) to verify the product's validity.
2. Invoke findProductOrThrow(id) to fetch the core entity data.
3. Map general attributes (title, dimensions, weight, price) to the ProductDetailResponseDto.
4. Append specialized data fields (e.g., authors for Books, trackList for CDs, director for DVDs) according to the specifications in Table B.

2. Product Update Logic (handleUpdate)

1. Retrieve the existing product record to obtain the originalValue.
2. Call validateUpdateData(originalPrice, currentPrice, dto).
 - Rule: The update is only permitted if: $0.3 \times \text{originalPrice} \leq \text{currentPrice} \leq 1.5 \times \text{originalPrice}$.
3. If the price is invalid, throw InvalidPriceException.

4. If valid, call trackChanges() to identify modified fields and saveHistoryLog() to record the "UPDATE" action.
5. Save the updated entity to the database.

3. Bulk Deletion/Deactivation (handleDeletion)

1. Iterate through the ids array (ensuring the batch size is ≤ 10).
2. For each ID, invoke checkDeletionConstraints():
 - Condition A (Stock > 0): Call deactivateProduct(id) to change the status to "deactivated". Physical deletion is not allowed.
 - Condition B (Stock == 0): Use validateStockQuantity() to ensure total deletions for the day ≤ 20 .
3. If Condition B is met, remove the record from the database.
4. Log every successful action via saveHistoryLog() with the specific reason.

4. New Product Creation (createProduct)

1. Receive the CreateProductDto and validate mandatory fields (e.g., barcode, title).
2. Verify that the barcode is unique within the system.
3. Initialize the product status as "active" and record the creation time.
4. Invoke saveHistoryLog(newId, "CREATE", "Initial product creation").
5. Persist the new product to the database and return the created object.

11. Design for class "CreateProductDto"

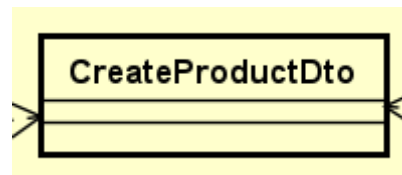


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	title	string	null	Name of the product (Required).
2	category	string	null	Category for filtering (Required).
3	type	string	null	Product type (Book, CD, DVD, Newspaper).

4	barcode	string	null	Unique identifier; must not exist in the system.
5	originalPrice	number	0	Price before tax/discounts (Must be \$> 0\$).
6	currentPrice	number	0	Selling price (Must satisfy 30% - 150% of original).
7	stockQuantity	number	0	Initial quantity available (Must be ≥ 0).
8	weight	number	0	Weight in grams (Required for shipping calculation).
9	dimensions	object	null	Includes Width, Height, Length (Required for shipping).
10	specificAttributes	Object/Map	null	Dynamic fields: Author (Book), Artist (CD), Director (DVD), etc.

Table 2. Operation design

#	Name	Return type	Description
1	None	N/A	DTOs are used only for data transfer between layers.

Method Design (Logic for DTO Role):

1. Input Collection: The ProductCreateComponent (Boundary) collects values from the UI form.
2. Data Encapsulation: These values are packed into the CreateProductDto object.

3. Cross-Layer Transfer: The DTO is sent as a payload to `ProductApiService.createProduct()`, which then delivers it to the Backend Controller and Service.
4. Validation Check: The Service layer extracts the `originalPrice` and `currentPrice` from this DTO to perform the $0.3 \times \text{originalPrice} \leq \text{currentPrice} \leq 1.5 \times \text{originalPrice}$ validation before saving.

12. Design for class "UpdateProductDto"

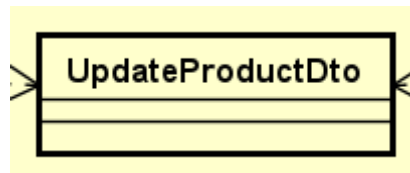


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	title	string	null	The updated name of the product.
2	category	string	null	The updated category for filtering.
3	currentPrice	number	null	The new selling price. Must follow the 30% - 150% rule.
4	weight	number	null	Updated weight for shipping recalculation.
5	dimensions	object	null	Updated height, width, and length.
6	specificDetails	object	null	Updated type-specific fields (e.g., tracklist, artists).

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	None	N/A	DTOs are data carriers and do not contain logic operations.

Parameter:

- None: This class does not contain operations.

Exception:

- None: This is a data structure; validation exceptions are handled by the ProductService when it processes this DTO.

Method Design (Logic/Algorithm):

- Controlled Update Role:
 1. The ProductEditComponent populates this DTO with values from the edit form.
 2. Security Constraint: The barcode field is strictly omitted from this DTO to ensure that the unique identifier of a product remains immutable once created.
 3. Business Validation: The currentPrice provided in this DTO is passed to the ProductService.validateUpdateData() method.
 4. The system ensures the new price remains within the legal bounds:
 $\$0.3 \times \text{originalPrice} \leq \text{currentPrice} \leq 1.5 \times \text{originalPrice}$
 5. History Tracking: This DTO is compared against the existing Product entity by the trackChanges() method to generate an audit log entry before the database is updated.

13. Design for class "ProductDetailResponseDto"



Table 1. Attribute design

#	Name	Data type	Default value	Description

1	title	string	null	The full title of the product.
2	category	string	null	Product category (e.g., Media, Electronics).
3	barcode	string	null	The unique identifier for the product.
4	originalPrice	number	0	The price of the product before tax.
5	currentPrice	number	0	The active selling price.
6	stockQuantity	number	0	Number of units available in the warehouse.
7	weight	number	0	Product weight in grams.
8	dimensions	object	null	Height, Width, and Length of the item.
9	imageUrl	string	null	URL for the product image display.
10	specificAttributes	Map<string, any>	null	Contains type-specific data (e.g., Author for Books, Tracklist for CDs).

Table 2. Operation design

#	Name	Return type	Description (purpose)

1	None	N/A	As a DTO, this class does not contain business logic or operations.
---	------	-----	---

Parameter:

- None: This class is a data carrier and does not have functional operations.

Exception:

- None: Since no logic is performed within this class, it does not trigger exceptions.

Method Design (Logic/Algorithm):

- **Data Aggregation and Construction:**
 1. The ProductService instantiates this DTO after a successful database query.
 2. It consolidates general attributes from the core Product entity with specialized data from sub-entities (Books, CDs, DVDs, or Newspapers) to satisfy the requirements of Table B in the specification.
- **Encapsulation for Data Flow:**
 1. The DTO serves as a secure data wrapper passed from the Service layer through the Controller and ApiService.
 2. It reaches the Boundary layer (ProductDetailPage) to be used for the final user display.
- **Dynamic UI Rendering Support:**
 1. The ProductDetailPage utilizes the specificAttributes map to render information dynamically.
 2. Based on the type field, the UI adapts to show relevant data (e.g., displaying a "Tracklist" only for CDs or "Director" for DVDs).
- **Network Efficiency:**
 1. This class ensures that all necessary detailed information is delivered in a single consolidated HTTP response, reducing the number of API calls required to view product details.

14. Design for class "ProductHistoryLog"

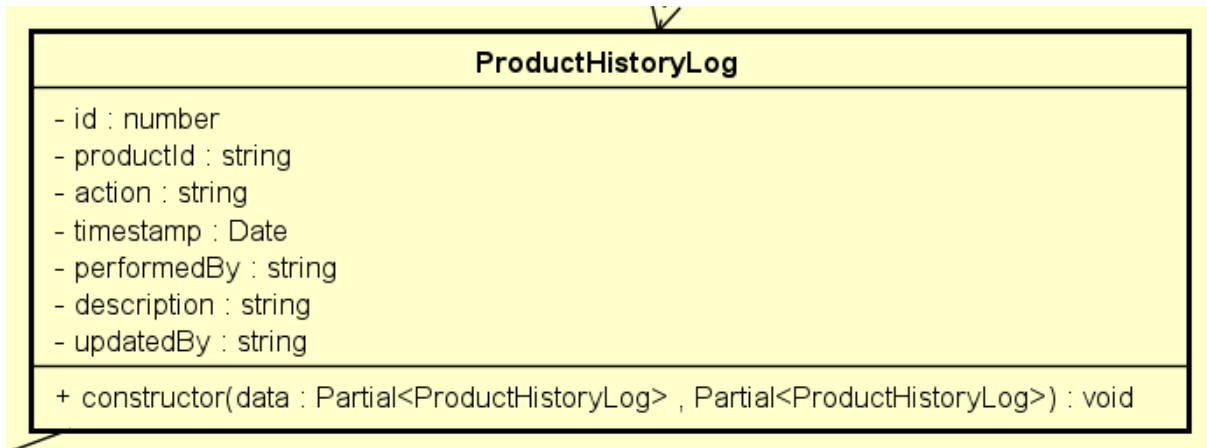


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	id	number	null	Internal unique identifier for the history log entry.
2	productId	string	null	The ID of the product that was modified or deleted.
3	action	string	null	The type of action performed (e.g., "CREATE", "UPDATE", "DELETE", "DEACTIVATE").
4	timestamp	Date	now()	The precise date and time when the action was recorded.
5	performedBy	string	null	The identity (username or ID) of the Product Manager who performed the action.
6	description	string	null	A detailed summary of the changes (e.g., "Price updated from 100k to 120k").

7	updatedBy	string	null	Redundant field or secondary modifier ID involved in the log event.
---	-----------	--------	------	---

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	constructor	void	Initializes a new history log instance with the provided event data.

Parameter:

- data: Partial<ProductHistoryLog> — A TypeScript partial object containing the initial set of values to populate the log entry.
- additionalData (second param): Partial<ProductHistoryLog> — Supplementary data used during the initialization process to compare states or add context.

Exception:

- None: This is a data entity class and does not contain complex logic that triggers exceptions.

Method Design (Logic/Algorithm):

Method: constructor(data, additionalData)

1. **Initialization:** The constructor receives two partial objects to allow flexible log creation.
2. **Mapping logic:**
 - It maps basic fields such as productId, action, and performedBy from the data parameter.
 - It utilizes the additionalData to enrich the log context, especially for complex updates.
3. **Automated Timestamping:** Inside the constructor, the timestamp attribute is explicitly assigned using the current system date/time (new Date()). This ensures that the audit record reflects the exact moment the administrative action occurred.
4. **Audit Integrity:** The description field is populated by the change-tracking logic. It specifically records the "Before" and "After" states of modified fields to satisfy the AIMS project.

5. **State Protection:** Once the object is initialized, the attributes are intended to be immutable (read-only) to prevent any tampering with the history logs after they are generated.

15. Design for class “InvoiceScreen”

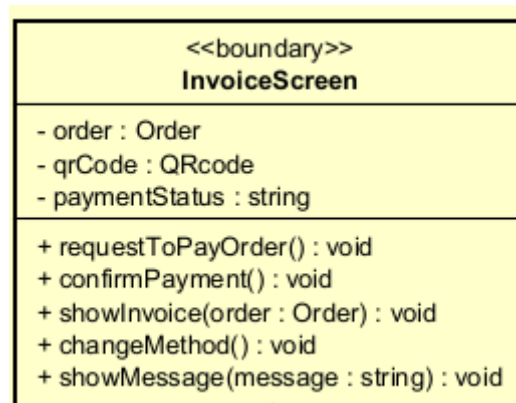


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- order	Order	null	Current order being paid and displayed on the screen.
2	- qrCode	QRcode	null	Generated VietQR information to be shown to the customer.
3	- paymentStatus	String	"UNPAID"	Latest payment status shown on the screen.
4	- paymentApiService	PaymentApiService	new PaymentApiService()	Facade used by the screen to call payment-related application logic.

Table 2. Operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	+ requestToPayOrder()	void	Start the QR payment flow for the current order.
2	+ confirmPayment()	void	Confirm the current order payment and update the UI accordingly.
3	+ showInvoice(order : Order)	void	Display invoice and order information on the screen.
4	+ changeMethod()	void	Switch from QR payment to the PayPal/credit-card flow.
5	+ showMessage(message : String)	void	Show a feedback message to the customer.

Operation: + requestToPayOrder()

Parameter: None. The method uses the current value of this.order.

Exception: PaymentException when the order is invalid or the QR request fails.

Method:

1. Check that this.order is not null and is ready for payment.
2. Call paymentApiService.requestQRCode(this.order).
3. Store the returned QRcode in this.qrCode.
4. Set paymentStatus to "PENDING" and refresh the invoice area.
5. Display the QR image and related transfer information to the customer.

Operation: + confirmPayment()

Parameter: None. The method uses the current value of this.order.

Exception: PaymentException when confirmation cannot be completed.

Method:

6. Call paymentApiService.confirmPayment(this.order).
7. Read the returned PaymentTransaction status.
8. Update paymentStatus based on the transaction result.
9. Show a success, failure, or cancellation message on the screen.
10. Keep the invoice visible so the calling use case can continue if payment succeeds.

Operation: + showInvoice(order : Order)

Parameter: order : Order

Exception: None.

Method:

11. Read payable information from the given order and its invoice data.
12. Render total amount, delivery address, and order identifier.
13. If a QRcode is already available, render the QR image together with bank information.

Operation: + changeMethod()

Parameter: None.

Exception: None.

Method:

14. Read the customer action requesting another payment method.
15. Notify the application layer to redirect to the PayPal/credit-card flow.
16. Update the screen so the customer can continue with the new method.

Operation: + showMessage(message : String)

Parameter: message : String

Exception: None.

Method:

17. Receive the message text from the calling logic.
18. Render the message in the notification area of the screen.

Keep the rest of the UI state unchanged

16. Design for class “PaymentApiService”

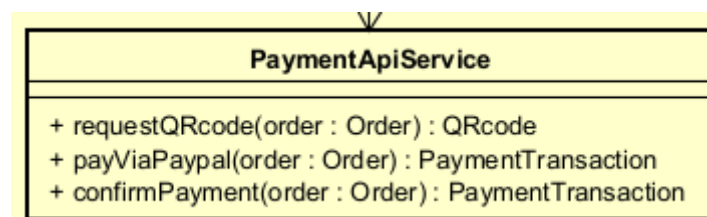


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- paymentController	PaymentController	new PaymentController()	Application-level controller coordinating the pay-order use case.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ requestQRCode(order : Order)	QRcode	Request a QR code for the given order.
2	+ payViaPayPal(order : Order)	void	Start payment through an alternative gateway.
3	+ confirmPayment(order : Order)	PaymentTransaction	Confirm the payment result for the given order.

Operation: + requestQRCode(order : Order)

Parameter: order : Order

Exception: PaymentException when the controller cannot obtain a QR code.

Method:

1. Validate that the supplied order is not null.
2. Delegate the request to paymentController.payOrder(order).
3. Return the QRcode object back to the boundary class.

Operation: + payViaPayPal(order : Order)

Parameter: order : Order

Exception: PaymentException when the alternative gateway cannot process the order.

Method:

4. Forward the order to paymentController.payViaPayPal(order).

5. Let the controller and gateway handle the payment flow.
6. Return control to the boundary class after the delegation finishes.

Operation: + confirmPayment(order : Order)

Parameter: order : Order

Exception: PaymentException when confirmation fails.

Method:

7. Delegate confirmation to paymentController.confirmPayment(order).
8. Return the confirmed PaymentTransaction to the screen.

17. Design for class “PaymentController”

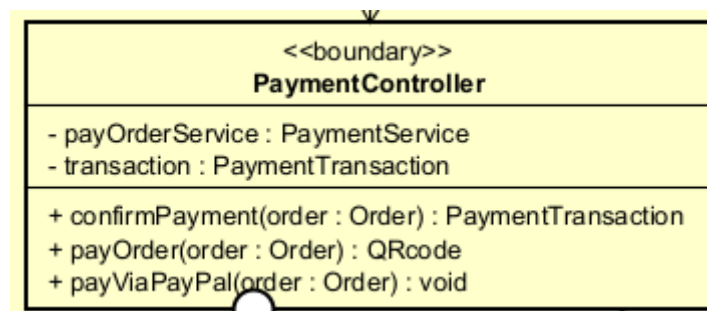


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- payOrderService	PaymentService	new PaymentService()	Core payment service for VietQR-related application logic.
2	- payThroughGatewayService	PayThroughGatewayService	new PayThroughGatewayService()	Service used when the customer switches to another

				payment gateway.
3	- transaction	PaymentTransaction	null	Latest transaction processed by the controller.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ confirmPayment(order : Order)	PaymentTransaction	Confirm VietQR payment for an order.
2	+ payOrder(order : Order)	QRcode	Generate a VietQR code for the order.
3	+ payViaPayPal(order : Order)	void	Pay the order through the configured alternative gateway.

Operation: + confirmPayment(order : Order)

Parameter: order : Order

Exception: PaymentException when the order cannot be confirmed.

Method:

1. Delegate the confirmation request to payOrderService.confirmPayment(order).
2. Store the returned transaction in this.transaction.
3. If the transaction is successful, update the order state for the calling use case.
4. Return the transaction to the caller.

Operation: + payOrder(order : Order)

Parameter: order : Order

Exception: PaymentException when QR generation fails.

Method:

5. Check that the order is valid for payment.
6. Request a QR code from payOrderService.requestQRCode(order).

7. Return the generated QRcode to the boundary/service facade.

Operation: + payViaPayPal(order : Order)

Parameter: order : Order

Exception: PaymentException when gateway payment fails.

Method:

8. Pass the order to payThroughGatewayService.payOrder(order).
9. Store the returned transaction in this.transaction.
10. Let the caller continue with the alternative payment flow.

18. Design for class “PaymentService”

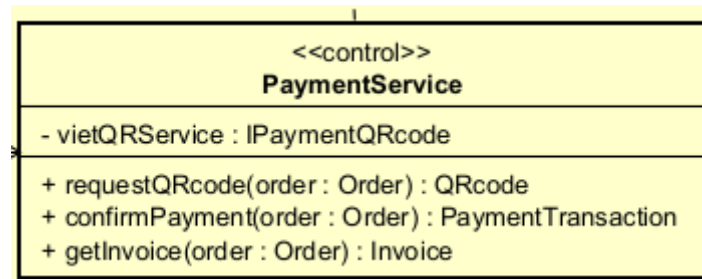


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- vietQRService	VietQRService	new VietQRService()	Service responsible for accessing VietQR APIs.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ requestQRCode(order : Order)	QRcode	Create a VietQR code for the order.
2	+ confirmPayment(order : Order)	PaymentTransaction	Check VietQR transaction status for the order.
3	+ getInvoice(order : Order)	Invoice	Build or retrieve the invoice shown during payment.

Operation: + requestQRCode(order : Order)

Parameter: order : Order

Exception: PaymentException when the order data or VietQR call is invalid.

Method:

1. Extract transfer data such as amount, content, and order identifier from the order.
2. Delegate QR creation to vietQRService.generateQRCode(order).
3. Return the resulting QRcode object.

Operation: + confirmPayment(order : Order)

Parameter: order : Order

Exception: PaymentException when VietQR status cannot be obtained.

Method:

4. Delegate status checking to vietQRService.checkPaymentStatus(order).
5. Return the resulting PaymentTransaction.

Operation: + getInvoice(order : Order)

Parameter: order : Order

Exception: None.

Method:

6. Read pricing and delivery information from the order.
7. Create or retrieve an Invoice object for display and later persistence.
8. Return the invoice to the caller.

19. Design for class “PayThroughGatewayService”

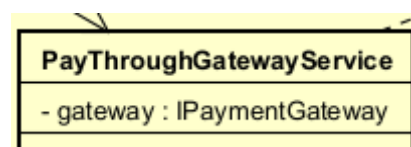


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- gateway	IPaymentGateway	null	Selected external payment gateway, such as PayPal.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ payOrder(order : Order)	PaymentTransaction	Submit the order to the configured external gateway.

Operation: + payOrder(order : Order)

Parameter: order : Order

Exception: PaymentException when no gateway is configured or the gateway rejects the payment.

Method:

1. Check that gateway is configured.
2. Delegate the request to gateway.payOrder(order).
3. Return the resulting PaymentTransaction to the caller.

20. Design for class “IPaymentGateway”

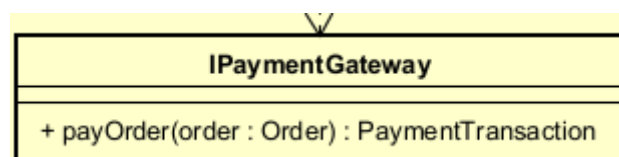


Table 1. Attribute design

#	Name	Data type	Default value	Description
-	(None)	N/A	N/A	This class/interface introduces no additional attributes in the current design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ payOrder(order : Order)	PaymentTransaction	Interface contract for processing a paid order through an external gateway.

Operation: + payOrder(order : Order)

Parameter: order : Order

Exception: PaymentException (implementation-specific).

Method:

1. Validate the order data required by the concrete gateway.
2. Submit the payment request to the gateway provider.
3. Return a PaymentTransaction describing the result.

21. Design for class “VietQRService”

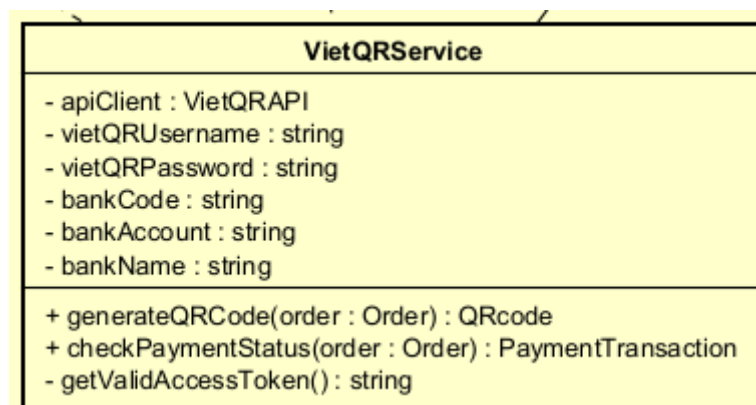


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- apiClient	VietQRAPI	new VietQRAPI()	Low-level API client used to call VietQR endpoints.
2	- vietQRUsername	String	""	Sandbox or production user name for VietQR authentication.

3	- vietQRPassword	String	""	Sandbox or production password for VietQR authentication.
4	- bankCode	String	""	Receiving bank code used in generated QR data.
5	- bankAccount	String	""	Receiving bank account number.
6	- bankName	String	""	Receiving account holder name.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ generateQRCode(order : Order)	QRcode	Generate VietQR data for the given order.
2	+ checkPaymentStatus(order : Order)	PaymentTransaction	Query VietQR transaction status for the order.
3	+ getValidAccessToken()	String	Obtain a valid VietQR access token before calling APIs.

Operation: + generateQRCode(order : Order)

Parameter: order : Order

Exception: InvalidTokenException or PaymentException when request creation or API invocation fails.

Method:

1. Call getValidAccessToken() to ensure a valid token is available.
2. Create a QRGenerateRequest from the order and configured bank information.
3. Build the VietQR request string.
4. Invoke apiClient.generateQRcode(requestString).
5. Return the parsed QRcode object.

Operation: + checkPaymentStatus(order : Order)

Parameter: order : Order

Exception: PaymentException when status synchronization fails.

Method:

6. Build the status-check request using the order identifier.
7. Invoke apiClient.checkPaymentStatus(requestString).
8. Map the API response to a PaymentTransaction.
9. Return the transaction result.

Operation: + getValidAccessToken()

Parameter: None.

Exception: InvalidTokenException when authentication cannot be completed.

Method:

10. Create a QRAccessTokenRequest with vietQRUsername and vietQRPassword.
11. Build the authorization request string or header.
12. Invoke apiClient.getAccessToken(requestString).
13. Return the access token string if it is valid and non-expired.

22. Design for class “VietQRAPI”

VietQRAPI	
- GET_TOKEN_URL : string - GENERATE_QR_URL : string - TEST_CALLBACK_URL : string - httpClient : HttpClient	
~ getAccessToken(requestString : string) : string ~ generateQRcode(requestString : string) : QRcode ~ checkPaymentStatus(requestString : string) : PaymentTransaction	

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- GET_TOKEN_URL	String	""	Endpoint used to request VietQR access tokens.
2	- GENERATE_QR_URL	String	""	Endpoint used to request QR code generation.

3	- TEST_CALLBACK_URL	String	""	Endpoint used to simulate or synchronize successful payments.
4	- httpClient	HttpClient	HttpClient.newHttpClient()	HTTP client used for outbound API calls.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	~ getAccessToken(requestString : String)	String	Call VietQR token API and return the raw access token.
2	~ generateQRcode(requestString : String)	QRcode	Call VietQR QR-generation API and return a QRcode object.
3	~ checkPaymentStatus(requestString : String)	PaymentTransaction	Call VietQR status API and return a payment transaction result.

Operation: ~ getAccessToken(requestString : String)

Parameter: requestString : String

Exception: UnknownException when the HTTP call fails or the response is malformed.

Method:

1. Send an HTTP request to GET_TOKEN_URL using the given request payload.
2. Read the response body from VietQR.
3. Return the raw token string or parsed token value to the caller.

Operation: ~ generateQRcode(requestString : String)

Parameter: requestString : String

Exception: UnknownException when the QR generation call fails.

Method:

4. Send an HTTP request to GENERATE_QR_URL using the given request payload.

5. Receive the VietQR response body.
6. Create a QRcode object and let it parse the response.
7. Return the QRcode object.

Operation: ~ checkPaymentStatus(requestString : String)

Parameter: requestString : String

Exception: UnknownException when the status call fails.

Method:

8. Send an HTTP request to TEST_CALLBACK_URL or the transaction-sync endpoint.
9. Read the response body returned by VietQR.
10. Map the response to a PaymentTransaction object.
11. Return the transaction.

23. Design for class “QRGenerateRequest”

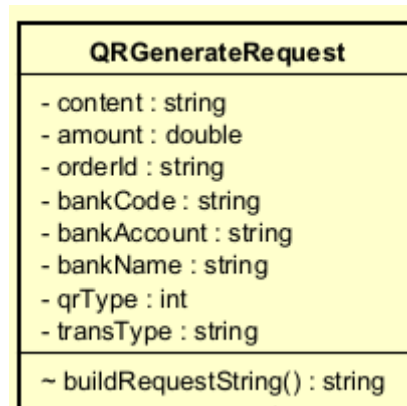


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- content	String	""	Transfer content shown to the customer and sent to VietQR.
2	- amount	double	0.0	Total amount to be paid.

3	- orderId	String	""	System order identifier used in the payment content.
4	- bankCode	String	""	Destination bank code.
5	- bankAccount	String	""	Destination bank account number.
6	- bankName	String	""	Destination account holder name.
7	- qrType	int	0	QR generation mode required by VietQR.
8	- transType	String	""	Transaction type sent to VietQR.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	~ buildRequestString()	String	Serialize request data into the format required by VietQR.

Operation: ~ buildRequestString()

Parameter: None.

Exception: None.

Method:

1. Read all internal request fields.
2. Validate mandatory fields such as amount, orderId, bankCode, and bankAccount.
3. Serialize the data into JSON or form-data expected by VietQR.
4. Return the resulting request string.

24. Design for class “QRAccessTokenRequest”

QRAccessTokenRequest
- userName : string - password : string
~ buildAuthorizationHeader() : string

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- userName	String	""	VietQR user name used for authentication.
2	- password	String	""	VietQR password used for authentication.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	~ buildAuthorizationHeader()	String	Build the authentication header or token request payload.

Operation: ~ buildAuthorizationHeader()

Parameter: None.

Exception: None.

Method:

1. Read userName and password.
2. Compose the authorization data in the format required by VietQR.
3. Return the resulting header or request string.

25. Design for class “QRAccessTokenResponse”

QRAccessTokenResponse
- accessToken : string - tokenType : string - expiresIn : int
~ parseResponseString(response : string) : void

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- accessToken	String	""	Access token returned by VietQR.
2	- tokenType	String	""	Token type returned by VietQR.
3	- expiresIn	int	0	Token lifetime in seconds.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	~ parseResponseString(response : String)	void	Parse a raw VietQR token response and populate this object.

Operation: ~ parseResponseString(response : String)

Parameter: response : String

Exception: UnknownException when the response cannot be parsed.

Method:

1. Read the raw response returned by the token API.
2. Extract accessToken, tokenType, and expiresIn values.
3. Assign the parsed values to the current object.

26. Design for class “Order”

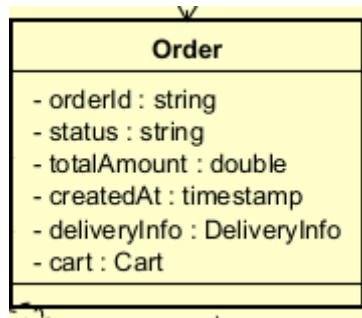


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- orderId	String	""	Unique identifier of the order.
2	- status	String	"CREATED"	Current lifecycle status of the order.
3	- totalAmount	double	0.0	Total amount the customer must pay.
4	- createdAt	LocalDateTime	LocalDateTime.now()	Timestamp when the order is created.
5	- deliveryInfo	DeliveryInfo	null	Delivery information associated with the order.
6	- cart	Cart	null	Shopping cart data used to create the order.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ validateForPayment()	boolean	Check whether the order is complete and payable.
2	+ updateStatus(status : String)	void	Update the order lifecycle status.

3	+ getTransferContent()	String	Return a compact transfer content string used for payment.
---	------------------------	--------	--

Operation: + validateForPayment()

Parameter: None.

Exception: None.

Method:

1. Check that orderId is present.
2. Check that totalAmount is greater than zero.
3. Check that deliveryInfo exists and is valid.
4. Return true only when the order is ready to be paid.

Operation: + updateStatus(status : String)

Parameter: status : String

Exception: None.

Method:

5. Validate that the new status is allowed by the order lifecycle.
6. Assign the new value to this.status.

Operation: + getTransferContent()

Parameter: None.

Exception: None.

Method:

7. Read the order identifier.
8. Convert it into the compact payment content required by the use case.
9. Return the resulting string.

27. Design for class “DeliveryInfo”

DeliveryInfo
- reveiverName : string - email : string - phoneNumber : string - address : string - province : string - deliveryNotes : string
+ validate() : boolean

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- receiverName	String	""	Full name of the receiver.
2	- email	String	""	Email address used for notifications and invoice sending.
3	- phoneNumber	String	""	Phone number of the receiver.
4	- address	String	""	Detailed delivery address.
5	- province	String	""	Province or city used for shipping-fee calculation.
6	- deliveryNotes	String	""	Additional delivery instructions.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ validate()	boolean	Validate required delivery fields before payment.

Operation: + validate()

Parameter: None.

Exception: None.

Method:

1. Check that receiverName, phoneNumber, address, and province are not blank.
2. Check that email and phone number formats are valid if provided.
3. Return true only when all required data are valid.

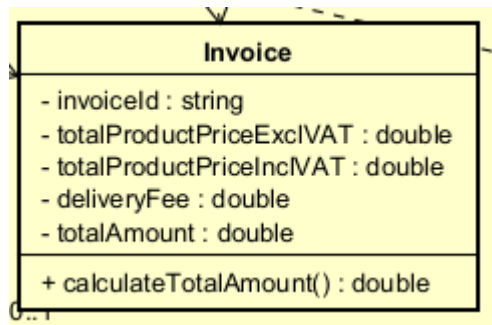
28. Design for class “Invoice”

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- invoiceId	String	""	Unique identifier of the invoice.
2	- totalProductPriceExclVAT	double	0.0	Total product price excluding VAT.
3	- totalProductPriceInclVAT	double	0.0	Total product price including VAT.
4	- deliveryFee	double	0.0	Shipping fee calculated for the order.
5	- totalAmount	double	0.0	Final amount to be paid.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ calculateTotalAmount()	double	Compute the final payable amount of the invoice.

Operation: + calculateTotalAmount()

Parameter: None.

Exception: None.

Method:

1. Read totalProductPriceInclVAT and deliveryFee.
2. Compute totalAmount = totalProductPriceInclVAT + deliveryFee.
3. Store the computed value in this.totalAmount.
4. Return totalAmount.

29. Design for class “QRcode”

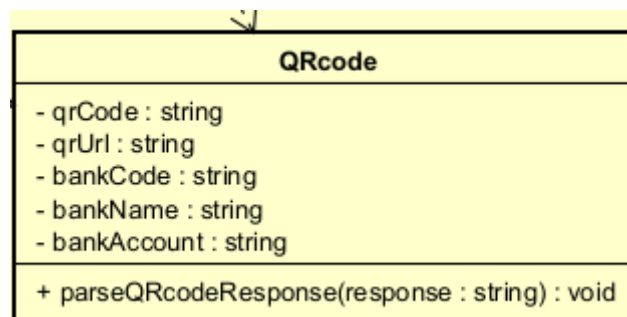


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- qrCode	String	""	Encoded QR content or identifier returned by VietQR.
2	- qrUrl	String	""	URL of the generated QR image.
3	- bankCode	String	""	Receiving bank code embedded in the QR data.
4	- bankName	String	""	Receiving bank name embedded in the QR data.
5	- bankAccount	String	""	Receiving bank account embedded in the QR data.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ parseQRCodeResponse(response : String)	void	Parse the raw QR-generation response returned by VietQR.

Operation: + parseQRCodeResponse(response : String)

Parameter: response : String

Exception: UnknownException when the response is malformed.

Method:

1. Read the raw response string returned by VietQR.
2. Extract QR image URL and banking fields.
3. Assign the parsed values to the current object.

30. Design for class “PaymentTransaction”

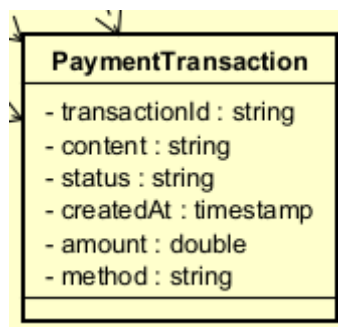


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- transactionId	String	""	Unique identifier of the payment transaction.
2	- content	String	""	Transaction content associated with the payment.
3	- status	String	"PENDING"	Current transaction status.

4	- createdAt	LocalDateTime	LocalDateTime.now() ()	Timestamp when the transaction is created or synchronized.
5	- amount	double	0.0	Paid amount.
6	- method	String	"VIETQR"	Payment method used for the transaction.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ isSuccessful()	boolean	Check whether the transaction has succeeded.
2	+ markStatus(status : String)	void	Update the status of the transaction.

Operation: + isSuccessful()

Parameter: None.

Exception: None.

Method:

1. Compare this.status with the success state defined by the payment flow.
2. Return true if the transaction is successful; otherwise return false.

Operation: + markStatus(status : String)

Parameter: status : String

Exception: None.

Method:

3. Validate the incoming status value.
4. Assign the value to this.status.

31. Design for class “PaymentException”

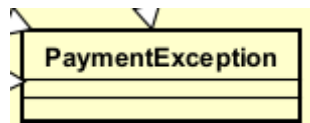


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- message	String	""	Human-readable explanation of the payment error.
2	- errorCode	String	"PAYMENT_ERROR"	Code used to classify the payment error.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ PaymentException(message : String)	N/A	Create an exception instance.
2	+ getUserMessage()	String	Return a message suitable for showing to the user.

Operation: + PaymentException(message : String)

Parameter: message : String

Exception: None.

Method:

1. Assign the given message to the current exception object.
2. Initialize the default errorCode if needed.

Operation: + getUserMessage()

Parameter: None.

Exception: None.

Method:

3. Read the internal message and error code.
4. Format a readable message for the UI or caller.
5. Return the formatted string.

32. Design for class “InvalidTokenException”



Table 1. Attribute design

#	Name	Data type	Default value	Description
-	(None)	N/A	N/A	This class/interface introduces no additional attributes in the current design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ InvalidTokenException(message : String)	N/A	Create an exception instance.

Operation: + InvalidTokenException(message : String)

Parameter: message : String

Exception: None.

Method:

1. Call the parent PaymentException constructor.
2. Set or preserve an error code indicating invalid token.

33. Design for class “UserCancelledException”

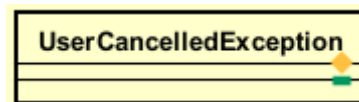


Table 1. Attribute design

#	Name	Data type	Default value	Description
-	(None)	N/A	N/A	This class/interface introduces no additional attributes in the current design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ UserCancelledException(message : String)	N/A	Create an exception instance.

Operation: + UserCancelledException(message : String)

Parameter: message : String

Exception: None.

Method:

1. Call the parent PaymentException constructor.
2. Set or preserve an error code indicating user cancellation.

34. Design for class “UnknownException”

Table 1. Attribute design

#	Name	Data type	Default value	Description
-	(None)	N/A	N/A	This class/interface introduces no additional attributes in the current design.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ UnknownException(message : String)	N/A	Create an exception instance.

Operation: + UnknownException(message : String)

Parameter: message : String

Exception: None.

Method:

1. Call the parent PaymentException constructor.
2. Set or preserve an error code indicating an unexpected failure.

35. Design for class “CartScreen”

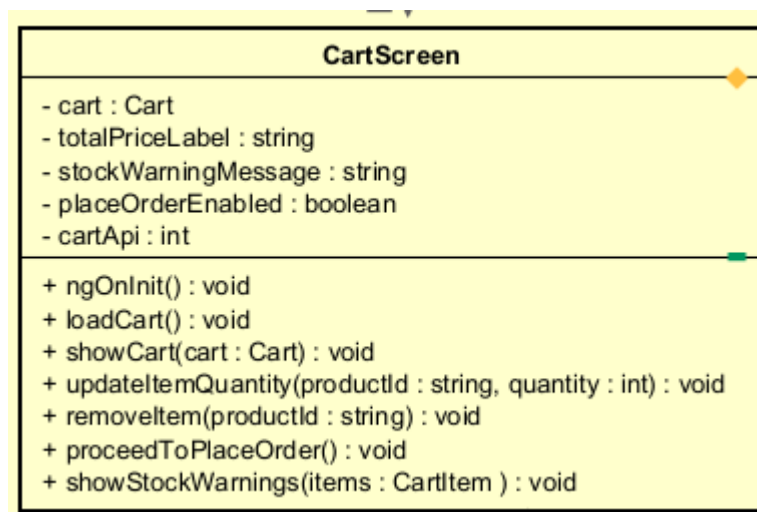


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- cart	Cart	null	Current cart currently displayed on the cart screen.

2	- totalPriceLabel	String	""	Rendered label showing the total product price excluding VAT.
3	- stockWarningMessage	String	""	Aggregated warning shown when some items do not have enough stock.
4	- placeOrderEnabled	boolean	false	Flag controlling whether the customer can continue to the Place Order flow.
5	- cartApiService	CartApiService	new CartApiService()	Boundary-level API facade used to call cart-related application services.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ ngOnInit()	void	Initialize the screen and trigger the first cart load.
2	+ loadCart()	void	Fetch the current cart and refresh the whole cart screen.
3	+ showCart(cart : Cart)	void	Display cart items and summary information on the UI.
4	+ updateItemQuantity(productId : String, quantity : int)	void	Handle a quantity-change request coming from the cart UI.
5	+ removeItem(productId : String)	void	Remove an item from the current cart and redraw the screen.
6	+ proceedToPlaceOrder()	void	Continue to the next use case step if the cart is valid for ordering.
7	+ showStockWarnings(items : List<CartItem>)	void	Show warnings for items that do not have enough stock.

Operation: + ngOnInit()

Parameter: None.

Exception: None directly; backend failures are converted into user-facing messages.

Method:

1. Initialize UI state and disable the Place Order button temporarily.
2. Call loadCart() to retrieve the current cart from the application layer.
3. Render the returned cart information on screen.
4. Update warnings and enablement state based on stock availability.

Operation: + loadCart()

Parameter: None.

Exception: ApplicationException when the cart cannot be retrieved from the backend.

Method:

1. Call cartApiService.getCurrentCart().
2. Store the returned Cart in this.cart.
3. Call showCart(this.cart) to render item rows and total price.
4. Call showStockWarnings(this.cart.getInsufficientItems()) and update placeOrderEnabled.

Operation: + showCart(cart : Cart)

Parameter: cart: Cart object that must be shown to the customer.

Exception: None.

Method:

1. Assign the input cart to this.cart.
2. Render a CartItemCard for each item in cart.items.
3. Update totalPriceLabel with cart.calculateTotalPriceExclVAT().
4. Refresh warning and Place Order controls based on current cart state.

Operation: + updateItemQuantity(productId : String, quantity : int)

Parameter: productId: identifier of the product to update; quantity: new requested quantity.

Exception: ValidationException when quantity is negative or productId is empty.

Method:

1. Validate that productId is not blank and quantity is greater than or equal to zero.
2. Call cartApiService.updateItemQuantity(productId, quantity).
3. Receive the updated Cart from the application layer.
4. Call showCart(updatedCart) to redraw the screen.

Operation: + removeItem(productId : String)

Parameter: productId: identifier of the product to remove.

Exception: ValidationException when productId is blank.

Method:

1. Validate the productId parameter.
2. Call cartApiService.removeItem(productId).
3. Receive the updated Cart after removal.
4. Call showCart(updatedCart) and refresh warnings.

Operation: + proceedToPlaceOrder()

Parameter: None. The operation uses the current value of this.cart.

Exception: BusinessException when the cart still contains insufficient-stock items.

Method:

1. Check whether this.cart is not null and canProceedToPlaceOrder() returns true.
2. If the cart is invalid, show a warning message and keep the user on the same screen.
3. If the cart is valid, navigate to the delivery-information / Place Order step.
4. Preserve the current cart state for the next screen.

Operation: + showStockWarnings(items : List<CartItem>)

Parameter: items: collection of cart items whose requested quantity exceeds the available stock.

Exception: None.

Method:

1. If the collection is empty, clear the warning area and enable place-order controls.
2. Otherwise, build a readable warning message for each insufficient item.
3. Store the rendered message in stockWarningMessage.
4. Disable Place Order until the cart becomes valid again.

36. Design for class “CartItemCard”

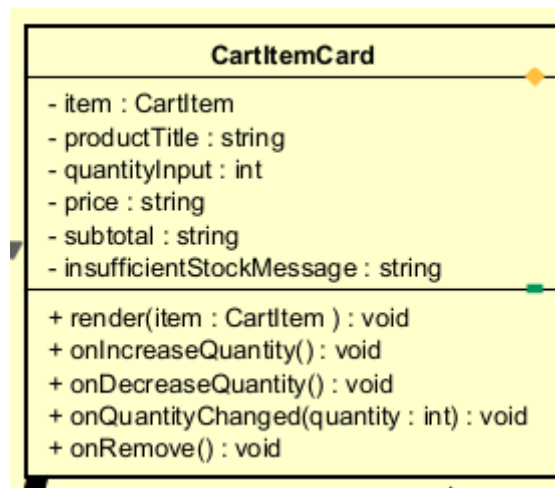


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- item	CartItem	null	Cart item represented by the current UI card.
2	- productTitle	String	""	Rendered title of the product shown in the item card.
3	- quantityInput	int	0	Current quantity value displayed in the quantity field.

4	- price	String	""	Display-friendly price of one product unit.
5	- subtotal	String	""	Display-friendly subtotal of the current cart item.
6	- insufficientStockMessage	String	""	Inline stock warning for this particular item card.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ render(item : CartItem)	void	Bind a CartItem to the card and update all displayed fields.
2	+ onIncreaseQuantity()	void	Increase the requested quantity by one and propagate the change.
3	+ onDecreaseQuantity()	void	Decrease the requested quantity by one and propagate the change.
4	+ onQuantityChanged(quantity : int)	void	Notify the cart screen that the quantity for this item has changed.
5	+ onRemove()	void	Request removal of this item from the cart.

Operation: + render(item : CartItem)

Parameter: item: cart item to be rendered.

Exception: None.

Method:

1. Assign the input item to this.item.
2. Read product title, quantity, unit price, and subtotal from the item and its product.
3. Populate the card labels and input widgets.
4. Show or clear insufficientStockMessage depending on item.hasInsufficientStock().

Operation: + onIncreaseQuantity()

Parameter: None.

Exception: None directly; validation is delegated to onQuantityChanged(int).

Method:

1. Read the current quantityInput value.
2. Increase it by one.
3. Call onQuantityChanged(quantityInput + 1).
4. Let the parent screen refresh using the returned updated cart.

Operation: + onDecreaseQuantity()

Parameter: None.

Exception: ValidationException when the quantity would become negative.

Method:

1. Read the current quantityInput value.
2. If the value is already zero, keep it unchanged.
3. Otherwise reduce it by one.
4. Call onQuantityChanged(new Value).

Operation: + onQuantityChanged(quantity : int)

Parameter: quantity: newly entered quantity for the represented product.

Exception: ValidationException when quantity is negative.

Method:

1. Validate the new quantity.
2. Update quantityInput with the validated value.
3. Publish the change event to CartScreen or call its updateItemQuantity(item.product.productId, quantity).
4. Wait for the screen to re-render this card with authoritative cart data.

Operation: + onRemove()

Parameter: None. The operation uses this.item.product.productId.

Exception: None.

Method:

1. Check that this.item is not null.
2. Extract the current productId from the bound item.
3. Notify CartScreen to remove that product from the cart.
4. Let the screen redraw the updated cart view.

37. Design for class “CartApiService”

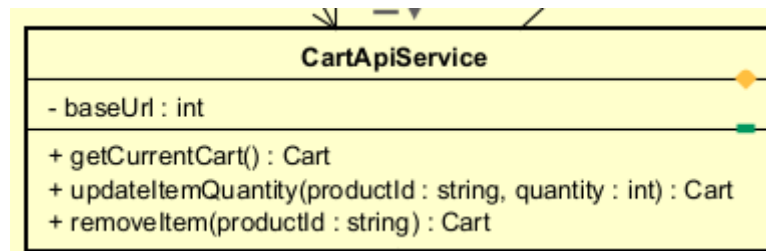


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- baseUrl	String	"/api/cart"	Base endpoint used by the boundary layer to access cart APIs.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ getCurrentCart()	Cart	Call the backend endpoint that returns the current cart.
2	+ updateItemQuantity(productId : String, quantity : int)	Cart	Invoke the API that updates a cart-item quantity and returns the refreshed cart.
3	+ removeItem(productId : String)	Cart	Invoke the API that removes one product from the cart.

Operation: + getCurrentCart()

Parameter: None.

Exception: ApplicationException when the API call fails or returns invalid data.

Method:

1. Build the HTTP GET request to baseUrl.
2. Send the request to the cart controller endpoint.
3. Map the response payload into a Cart object.
4. Return the mapped Cart to the calling boundary class.

Operation: + updateItemQuantity(productId : String, quantity : int)

Parameter: productId: identifier of the product; quantity: new requested quantity.

Exception: ValidationException or ApplicationException depending on the API response.

Method:

1. Validate the outgoing parameters.
2. Build the HTTP PUT/PATCH request to the quantity-update endpoint.
3. Submit productId and quantity to the backend.
4. Parse the returned Cart and pass it back to the caller.

Operation: + removeItem(productId : String)

Parameter: productId: identifier of the product to remove.

Exception: ValidationException or ApplicationException depending on the API response.

Method:

1. Validate that productId is provided.
2. Build the HTTP DELETE request for the specific cart item.
3. Call the backend endpoint.
4. Parse and return the refreshed Cart.

38. Design for class “CartController”

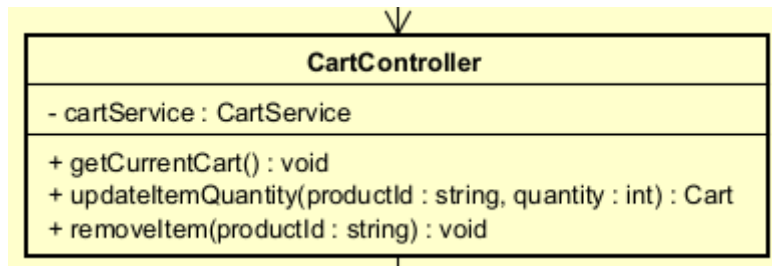


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- cartService	CartService	new CartService()	Application controller dependency used to perform cart-related use-case logic.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ getCurrentCart()	Cart	Receive a cart-read request from the API layer and delegate it to CartService.
2	+ updateItemQuantity(productId : String, quantity : int)	Cart	Handle a quantity-update request and return the refreshed cart.
3	+ removeItem(productId : String)	Cart	Handle a remove-item request and return the refreshed cart.

Operation: + getCurrentCart()

Parameter: None.

Exception: ApplicationException when the current cart cannot be obtained.

Method:

1. Receive the incoming request from CartApiService.
2. Call `cartService.getCurrentCart()`.
3. Return the obtained Cart to the API layer.

4. Let the boundary render the returned cart.

Operation: + updateItemQuantity(productId : String, quantity : int)

Parameter: productId: identifier of the product; quantity: requested quantity.

Exception: ValidationException when the request is malformed.

Method:

1. Validate request parameters.
2. Call cartService.updateItemQuantity(productId, quantity).
3. Receive the updated Cart from the service layer.
4. Return the updated Cart to the caller.

Operation: + removeItem(productId : String)

Parameter: productId: identifier of the product to remove.

Exception: ValidationException when productId is blank.

Method:

1. Validate the incoming productId.
2. Call cartService.removeItem(productId).
3. Receive the updated Cart after removal.
4. Return that Cart to the API layer.

39. Design for class “CartService”

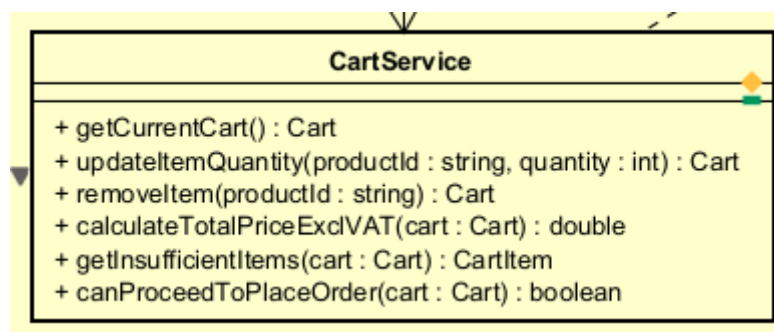


Table 1. Attribute design

#	Name	Data type	Default value		Description
1	(none)	-	-		This service is treated as stateless in the provided diagram; it coordinates entity behavior rather than storing persistent UI state.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ getCurrentCart()	Cart	Return the current in-memory/session cart for the active customer flow.
2	+ updateItemQuantity(productId : String, quantity : int)	Cart	Update one item quantity in the current cart and return the refreshed cart.
3	+ removeItem(productId : String)	Cart	Remove one item from the current cart and return the refreshed cart.
4	+ calculateTotalPriceExclVAT(cart : Cart)	double	Calculate the total product price excluding VAT for a given cart.
5	+ getInsufficientItems(cart : Cart)	List<CartItem>	Return all cart items whose requested quantity exceeds current stock.
6	+ canProceedToPlaceOrder(cart : Cart)	boolean	Determine whether the cart is valid to continue to the Place Order use case.

Operation: + getCurrentCart()

Parameter: None.

Exception: ApplicationException when no active cart can be found.

Method:

1. Locate the current cart from the session or cart repository.

2. Ensure that derived values can be recalculated on demand.
3. Return the Cart to the controller.

Operation: + updateItemQuantity(productId : String, quantity : int)

Parameter: productId: identifier of the product; quantity: new quantity.

Exception: ValidationException when the new quantity is invalid.

Method:

1. Load the current cart.
2. Call cart.updateItemQuantity(productId, quantity).
3. Recalculate or re-read derived values such as total price and insufficient items.
4. Return the updated Cart.

Operation: + removeItem(productId : String)

Parameter: productId: identifier of the product to remove.

Exception: ValidationException when productId is blank.

Method:

1. Load the current cart.
2. Call cart.removeItem(productId).
3. Refresh derived values and validity checks.
4. Return the updated Cart.

Operation: + calculateTotalPriceExclVAT(cart : Cart)

Parameter: cart: cart whose amount is to be calculated.

Exception: None.

Method:

1. Iterate through all cart items.
2. Sum each item subtotal without adding VAT.
3. Return the calculated total amount.

Operation: + getInsufficientItems(cart : Cart)

Parameter: cart: cart to inspect.

Exception: None.

Method:

1. Iterate through all items contained in the cart.
2. For each item, call hasInsufficientStock().
3. Collect the items that fail the stock check.
4. Return the resulting collection.

Operation: + canProceedToPlaceOrder(cart : Cart)

Parameter: cart: cart to validate.

Exception: None.

Method:

1. Check that the cart exists and is not empty.
2. Check that getInsufficientItems(cart) returns an empty collection.
3. Return true only if all validations pass.

40. Design for class “Cart”

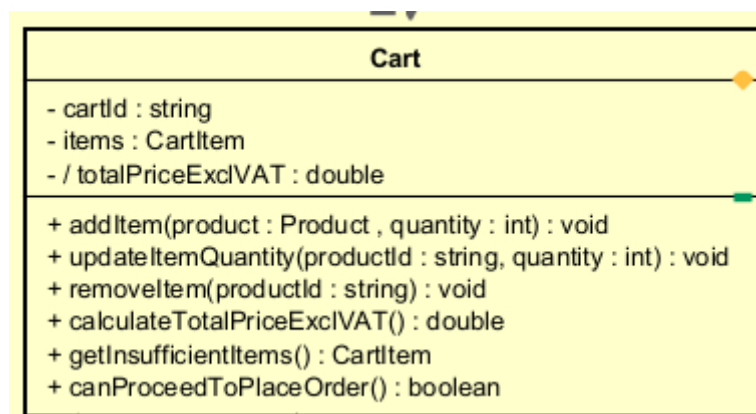


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	- cartId	String	generated id	Identifier used to distinguish the current cart.
2	- items	List<CartItem>	empty list	Collection of items currently contained in the cart.
3	/ totalPriceExclVAT	double	derived	Derived total amount of all products in the cart, excluding VAT.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ addItem(product : Product, quantity : int)	void	Add a product to the cart or merge quantities if it already exists.
2	+ updateItemQuantity(productId : String, quantity : int)	void	Change the quantity of one product in the cart.
3	+ removeItem(productId : String)	void	Remove one item from the cart.
4	+ calculateTotalPriceExclVAT()	double	Compute the sum of all cart-item subtotals excluding VAT.
5	+ getInsufficientItems()	List<CartItem>	Return all items whose requested quantity is greater than the available stock.
6	+ canProceedToPlaceOrder()	boolean	Check whether the cart is non-empty and contains no insufficient-stock item.

Operation: + addItem(product : Product, quantity : int)

Parameter: product: product to add; quantity: requested quantity.

Exception: ValidationException when product is null or quantity is not positive.

Method:

1. Validate input parameters.

2. Check whether the product already exists in items.
3. If it exists, increase the existing quantity; otherwise create a new CartItem.
4. Keep derived totals available for recalculation.

Operation: + updateItemQuantity(productId : String, quantity : int)

Parameter: productId: product to update; quantity: new quantity.

Exception: ValidationException when the product does not exist or quantity is invalid.

Method:

1. Find the CartItem whose product.productId matches productId.
2. If quantity is zero, remove the item; otherwise assign the new quantity.
3. Leave derived values to be recalculated on the next read.

Operation: + removeItem(productId : String)

Parameter: productId: identifier of the product to remove.

Exception: ValidationException when the product cannot be found.

Method:

1. Search the items collection for the matching productId.
2. Remove the matching CartItem from the collection.
3. Preserve collection consistency after removal.

Operation: + calculateTotalPriceExclVAT()

Parameter: None.

Exception: None.

Method:

1. Initialize total to zero.
2. Iterate over all items in the cart.
3. Add item.calculateSubTotal() for each element.
4. Return the resulting total.

Operation: + getInsufficientItems()

Parameter: None.

Exception: None.

Method:

1. Create an empty result collection.
2. Iterate through items.
3. Append any item for which hasInsufficientStock() is true.
4. Return the result collection.

Operation: + canProceedToPlaceOrder()

Parameter: None.

Exception: None.

Method:

1. Check that items is not empty.
2. Check that getInsufficientItems() is empty.
3. Return true only when both conditions are satisfied.

41. Design for class “CartItem”

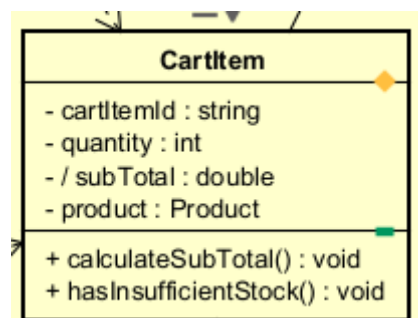


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- cartItemId	String	generated id	Identifier used for the cart-item instance.

2	- quantity	int	1	Requested quantity of the referenced product.
3	/ subTotal	double	derived	Derived subtotal equal to $\text{product.currentPrice} \times \text{quantity}$.
4	- product	Product	null	Referenced product being purchased in this cart item.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ calculateSubTotal()	double	Compute the subtotal contributed by this item.
2	+ hasInsufficientStock()	boolean	Check whether the requested quantity exceeds the available stock of the product.

Operation: + calculateSubTotal()

Parameter: None.

Exception: None.

Method:

1. Read product.currentPrice.
2. Multiply the unit price by quantity.
3. Return the computed subtotal.

Operation: + hasInsufficientStock()

Parameter: None.

Exception: None.

Method:

1. Read product.stockQuantity.
2. Compare quantity against the available stock.

3. Return true if quantity is greater than stockQuantity; otherwise return false.

42. Design for class “Product”

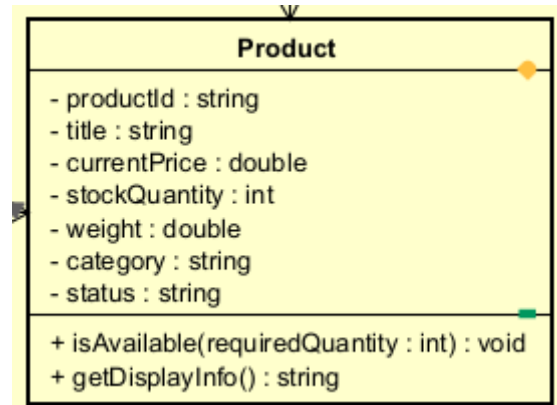


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- productId	String	generated id	Unique identifier of the product.
2	- title	String	""	Display title of the physical-media product.
3	- currentPrice	double	0.0	Current selling price of the product, excluding VAT.
4	- stockQuantity	int	0	Current available inventory quantity.
5	- weight	double	0.0	Weight of the product, used later for delivery-fee calculation.
6	- category	String	""	Product category such as book, newspaper, CD, or DVD.
7	- status	String	"ACTIVE"	Availability status of the product in the store catalog.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ isAvailable(requiredQuantity : int)	boolean	Check whether the product can satisfy the requested quantity.
2	+ getDisplayInfo()	String	Build a short text representation of the product for presentation layers.

Operation: + isAvailable(requiredQuantity : int)

Parameter: requiredQuantity: number of units requested by the cart.

Exception: ValidationException when requiredQuantity is negative.

Method:

1. Validate the input quantity.
2. Check that status allows selling.
3. Compare stockQuantity with requiredQuantity.
4. Return true only when the product is sellable and the stock is sufficient.

Operation: + getDisplayInfo()

Parameter: None.

Exception: None.

Method:

1. Read the product title, category, and current price.
2. Format them into a concise display string.
3. Return the generated string.

43. Design for class “UserManagementPage”

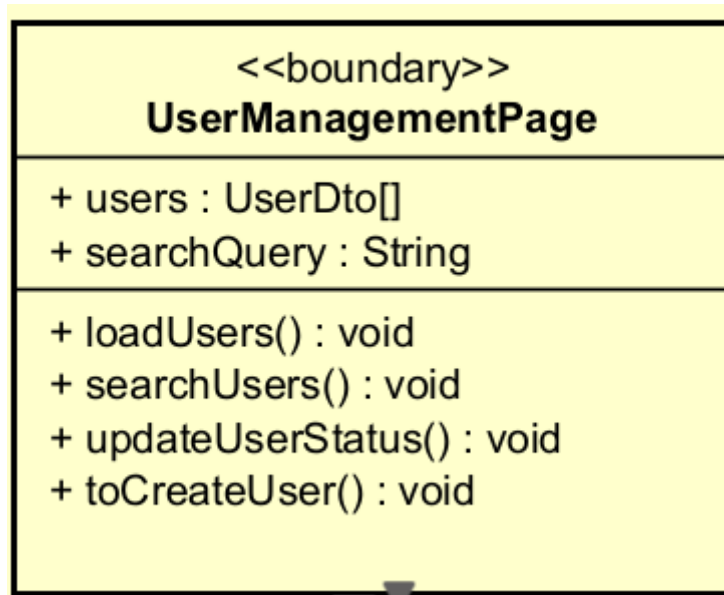


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	users	UserDto[]	[]	Stores the list of user data to be displayed in the table.
2	searchQuery	String	""	Stores the query string or keyword entered by the user for searching.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	loadUsers	void	Sends a request to load the user list from the system and display it on the interface.
2	searchUser	void	Triggers the filtering function for the user list based on searchQuery.
3	updateUserStatus	void	Handles the event of updating the status of a specific user, such as blocking or unblocking.
4	toCreateUser	void	Navigates the system to the create new user screen.

Detailed specification of main operations

1. loadUsers()

Parameter:

- None

Exception:

- APIException if the user list cannot be loaded
- NetworkException if the request to the server fails

Method

- Call UserAPIService.getUsers(searchQuery)
- Receive the list of users from the system
- Assign the result to the users attribute
- Display the user list on the interface

Other diagrams

- Sequence diagram if the interaction with UserAPIService is shown

2. searchUsers()

Parameter:

- None

Exception:

- ValidationException if searchQuery is invalid
- APIException if the search request fails

Method

- Read the current value of searchQuery
- Send the search request to UserAPIService.getUsers(searchQuery)
- Receive the search result
- Update the users list
- Refresh the interface

Other diagrams

- Activity diagram / sequence diagram if the searching flow is shown in detail

3. updateUserStatus()

Parameter:

- None

Exception:

- ValidationException if the selected user data is invalid
- APIException if the update request fails

Method

- Identify the user whose status needs to be updated
- Send the updated data to UserAPIService.updateUserStatus(data)
- Receive the response from the system
- Refresh the user list if the operation is successful

Other diagrams

- Sequence diagram if the status update flow is shown

4. toCreateUser()

Parameter:

- None

Exception:

- None

Method

- Navigate from the user management page to the user creation page
- Open UserFormPage so that the administrator can enter new user information

Other diagrams

- Sequence diagram if the navigation flow is shown

44. Design for class “UserFormPage”

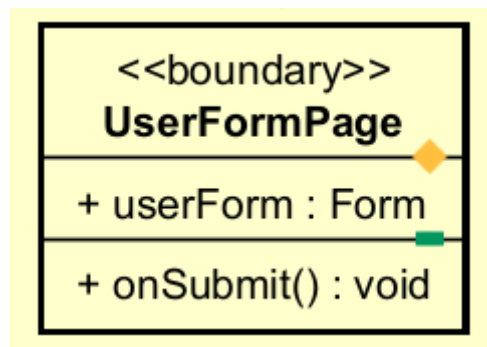


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	userForm	Form	null	Form object that manages the state and data of input fields.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	onSubmit	void	Captures the form submission event, validates the data, and calls the API to save it.
---	----------	------	---

Detailed specification of main operations

1. onSubmit()

Parameter:

- None

Exception:

- ValidationException if the form data is invalid
- APIException if the create user request fails

Method

- Validate the data in userForm
- If the data is valid, extract the form data
- Send the data to UserAPIService.createUser(data)
- Receive the result from the system
- Notify the user whether the operation is successful or not

Other diagrams

- Activity diagram / sequence diagram if the form submission flow is shown

45. Design for class “AdminActionLogPage”

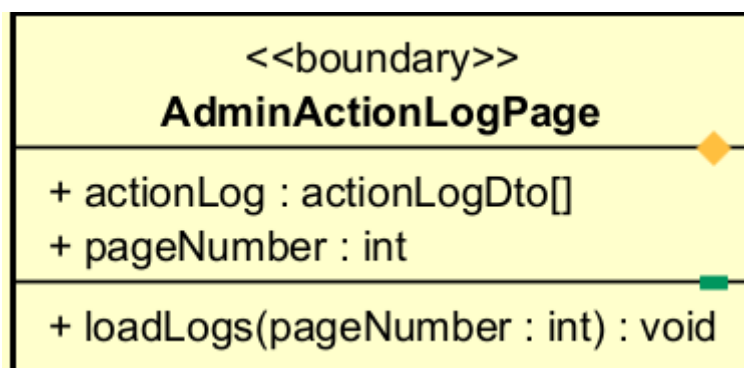


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	actionLog	ActionLogDto []	[]	Stores the array of system log data loaded for display.
2	pageNumber	int	1	Stores the current page number used for pagination.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadLogs	void	Loads the system log list based on the specified page number.

Detailed specification of main operations

1. loadLogs(pageNumber : int)

Parameter:

- pageNumber: page number of the action log list to be loaded

Exception:

- ValidationException if pageNumber <= 0
- APIException if the log loading request fails

Method

- Validate the value of pageNumber
- Send the request to UserAPIService.getActionLogs(pageNumber)
- Receive the log data from the system
- Assign the result to actionLog

- Update the interface to display the log list

Other diagrams

- Sequence diagram if the interaction with UserAPIService is shown

46. Design for class “LoginPage”

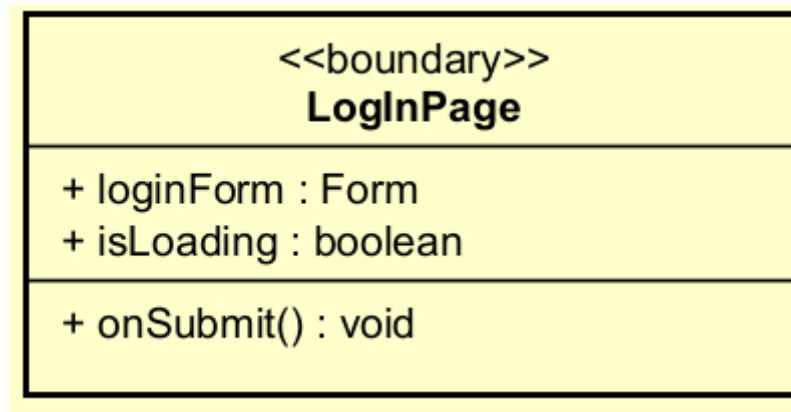


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	loginForm	Form	null	Manages the login form, including email and password fields.
2	isLoading	boolean	false	Indicates whether the system is currently waiting for authentication.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	onSubmit	void	Handles form submission, calls the authentication API, and displays the loading state.

Detailed specification of main operations

1. onSubmit()

Parameter:

- None

Exception:

- ValidationException if login credentials are invalid
- AuthException if authentication fails
- NetworkException if the server request fails

Method

- Validate the data in loginForm
- If valid, set isLoading = true
- Send the login credentials to AuthAPIService.login(credentials)
- Receive the authentication result from the system
- If login is successful, navigate to the appropriate page
- Set isLoading = false after the process is completed

Other diagrams

- Activity diagram / sequence diagram if the login flow is shown

47. Design for class “UserAPIService”

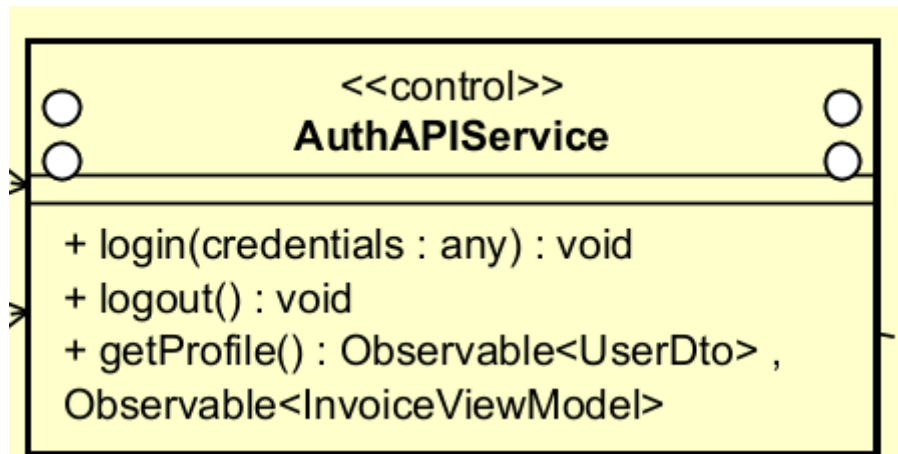


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	None			This service class does not define internal attributes in the class diagram.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getUsers(searchQuery : String)	Observable<UserDto[]>	Retrieve the list of users based on the search query
2	createUser(data : any)	Observable<UserDto>	Create a new user

3	getActionLogs(pageNumber : int)	Observable<actionLogDto[]>	Retrieve the action log list by page number
4	updateUserStatus(data : UserDto)	void	Update the status of a user

Detailed specification of main operations

1. getUsers(searchQuery : String)

Parameter:

- searchQuery: keyword used to search for users

Exception:

- APIException if the request fails
- NetworkException if the server cannot be reached

Method

- Send a request to the backend to retrieve the user list
- Pass searchQuery as the filter condition
- Receive the response as Observable<UserDto[]>
- Return the result to the caller

Other diagram

- Sequence diagram if the API interaction is shown

2. createUser(data : any)

Parameter:

- data: information of the new user to be created

Exception:

- ValidationException if the input data is invalid
- APIException if the create request fails

Method

- Send the new user data to the backend
- Receive the response from the system
- Return the result as Observable<UserDto>

Other diagrams

- Sequence diagram if the API call is shown
- 3. getActionLogs(pageNumber : int)**

Parameter:

- pageNumber: page number of the action log list to retrieve

Exception:

- ValidationException if pageNumber <= 0
- APIException if the request fails

Method

- Send a request to the backend to retrieve the action log list
- Pass pageNumber as the pagination parameter
- Receive the response as Observable<actionLogDto[]>
- Return the result to the caller

Other diagrams

- Sequence diagram if the API call is shown
- 4. updateUserStatus(data : UserDto)**

Parameter:

- data: user data whose status needs to be updated

Exception:

- ValidationException if data is invalid
- APIException if the update request fails

Method

- Send the user status update data to the backend
- Perform the update request
- No direct return data is shown in the current class diagram

Other diagrams

- Sequence diagram if the API interaction is shown

48. Design for class “AuthAPIService”

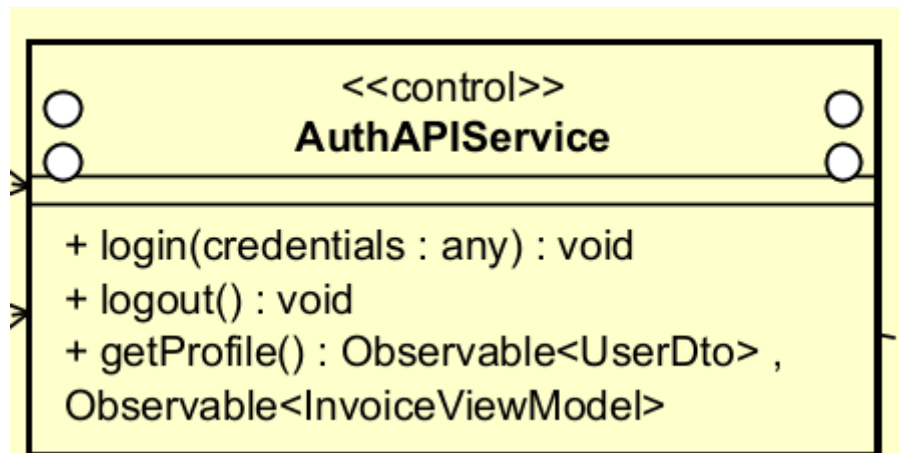


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	None			This service class does not define internal attributes in the class diagram

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	login(credentials : any)	void	Send login credentials to the authentication system
2	logout()	void	Log the user out of the system

3	getProfile()	Observable<User Dto>	Retrieve the current user's profile information
---	--------------	----------------------	---

Detailed specification of main operations

1. login(credentials : any)

Parameter:

- credentials: login information, usually including username/email and password

Exception:

- ValidationException if credentials are invalid
- AuthException if authentication fails
- NetworkException if the server request fails

Method

- Send the login credentials to the backend
- The system authenticates the account
- Receive the authentication result
- Store session information or token if login is successful

Other diagrams

- Sequence diagram if the authentication flow is shown

2. logout()

Parameter:

- None

Exception:

- APIException if the logout request fails

Method

- Send a logout request to the backend
- Clear local session information if necessary
- Redirect the user to the login page or unauthenticated state

Other diagrams

- Sequence diagram if the logout flow is shown

3. getProfile()

Parameter:

- None

Exception:

- AuthException if the current user is not authenticated
- APIException if the profile request fails

Method

- Send a request to the backend to retrieve the current user's profile
- Receive the response as Observable<UserDto>
- Return the result to the caller

Other diagrams

- Sequence diagram if the profile retrieval flow is shown

49. Design for class “UserController”

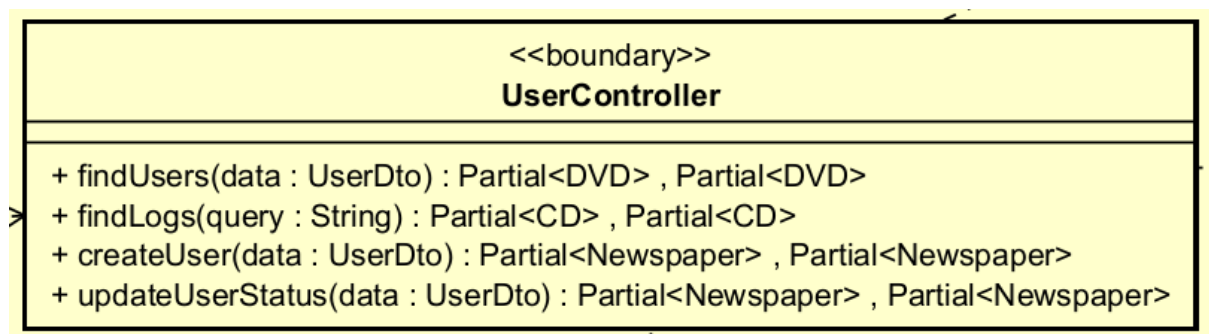


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	injected services	Service instances	injected	Contains service instances injected through Dependency Injection.
---	-------------------	-------------------	----------	---

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	findUsers(data : UserDto)	UserDto[]	Find users based on the given search/filter data
2	findLogs(query : String)	ActionLogDto[]	Find action logs based on the given query
3	createUser(data : UserDto)	UserDto	Create a new user
4	updateUserStatus(data : UserDto)	UserDto	Update the status of an existing user

Detailed specification of main operations

1. findUsers(data : UserDto)

Parameter:

- data: filtering information used to search for users

Exception:

- ValidationException if the input data is invalid
- UserException if the user retrieval process fails

Method

- Receive the request from the frontend
- Validate the input data
- Call `UserService.findUsers(data)`
- Return the list of matched users as `UserDto[]`

Other diagrams

- Sequence diagram if the request-processing flow is shown

2. findLogs(query : String)

Parameter:

- query: search condition used to retrieve action logs

Exception:

- `ValidationException` if query is invalid
- `UserException` if the log retrieval process fails

Method

- Receive the query string from the caller
- Validate the query
- Call `UserService.findLogs(query)`
- Return the result as `ActionLogDto[]`

Other diagrams

- Sequence diagram if the interaction with `UserService` is shown

3. createUser(data : UserDto)

Parameter:

- data: user information used to create a new user

Exception:

- `ValidationException` if the input data is incomplete or invalid
- `UserException` if the user creation process fails

Method

- Receive the user creation request
- Validate the input data
- Call `UserService.createUser(data)`

- Return the created user as UserDto

Other diagrams

- Sequence diagram if the creation flow is shown
4. **updateUserStatus(data : UserDto)**

Parameter:

- data: user data containing the updated status

Exception:

- ValidationException if the input data is invalid
- UserException if the update process fails

Method

- Receive the status update request
- Validate the input data
- Call UserService.updateUserStatus(data)
- Return the updated user information as UserDto

Other diagrams

- Sequence diagram if the update flow is shown

50. Design for class “UserService”

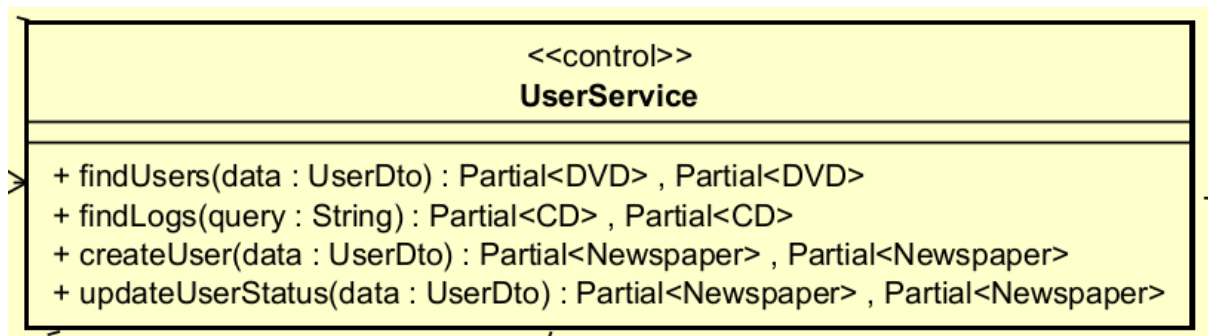


Table 1. Example of attribute design

#	Name	Data type	Default value	Description

1	None specified			This service class does not define internal attributes in the current class diagram
---	----------------	--	--	---

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	findUsers(data : UserDto)	UserDto[]	Process the business logic for retrieving users
2	findLogs(query : String)	ActionLogDto[]	Process the business logic for retrieving action logs
3	createUser(data : UserDto)	UserDto	Process the business logic for creating a new user
4	updateUserStatus(data : UserDto)	UserDto	Process the business logic for updating user status

Detailed specification of main operations

1. findUsers(data : UserDto)

Parameter:

- data: filtering data used to retrieve users

Exception:

- ValidationException if the input data is invalid
- UserException if no valid result can be retrieved

Method

- Validate the input data
- Query the data source for users matching the given conditions
- Convert the result into UserDto[]
- Return the user list to the controller

Other diagrams

- Sequence diagram / activity diagram if the service workflow is shown

2. **findLogs(query : String)**

Parameter:

- query: condition used to search action logs

Exception:

- ValidationException if query is invalid
- UserException if the log retrieval process fails

Method

- Validate the query
- Query the action log data source
- Convert the result into ActionLogDto[]
- Return the log list to the controller

Other diagrams

- Sequence diagram if the service flow is shown

3. **createUser(data : UserDto)**

Parameter:

- data: information of the user to be created

Exception:

- ValidationException if required fields are missing or invalid
- UserException if the user cannot be created

Method

- Validate the input user data
- Check business rules such as uniqueness or role assignment
- Create a new User entity
- Persist the entity to the data source

- Convert the created entity into UserDto
- Return the result to the controller

Other diagrams

- Activity diagram / sequence diagram if the creation logic is shown

4. updateUserStatus(data : UserDto)

Parameter:

- data: user data containing the new status

Exception:

- ValidationException if the input data is invalid
- UserException if the user is not found or cannot be updated

Method

- Validate the input data
- Find the target user
- Update the user's status
- Save the updated user data
- Convert the result into UserDto
- Return the updated data to the controller

Other diagrams

- Sequence diagram if the update flow is shown

51. Design for class “AuthController”

<<boundary>> AuthController
+ login(data : LoginDto, res : Response) : void + logout(res : Response) : void + getProfile(req : Request) : Partial<Newspaper> , Partial<Newspaper>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	None specified			This controller class does not define internal attributes in the current class diagram

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	login(data : LoginDto, res : Response)	void	Authenticate the user and return login response data
2	logout(res : Response)	void	Log the user out of the system
3	getProfile(req : Request)	UserDto	Retrieve the profile of the currently authenticated user

Detailed specification of main operations

1. login(data : LoginDto, res : Response)

Parameter:

- data: login credentials submitted by the user
- res: HTTP response object used to return the login result

Exception:

- ValidationException if login data is invalid
- AuthException if authentication fails

Method

- Receive login credentials from the client
- Validate the input data
- Call AuthService.validateUser(data)
- If valid, call AuthService.login(data)
- Write the authentication result to res

Other diagrams

- Sequence diagram if the authentication flow is shown

2. logout(res : Response)

Parameter:

- res: HTTP response object

Exception:

- AuthException if logout fails

Method

- Invalidate the current login session or token
- Clear authentication-related data
- Return the logout response through res

Other diagrams

- Sequence diagram if the logout flow is shown

3. getProfile(req : Request)

Parameter:

- req: HTTP request object containing user authentication information

Exception:

- AuthException if the request is not authenticated
- UserException if the profile cannot be retrieved

Method

- Extract the current user identity from req
- Validate the authentication state
- Retrieve the current user's profile
- Return the profile data as UserDto

Other diagrams

- Sequence diagram if the profile retrieval flow is shown

52. Design for class “AuthService”

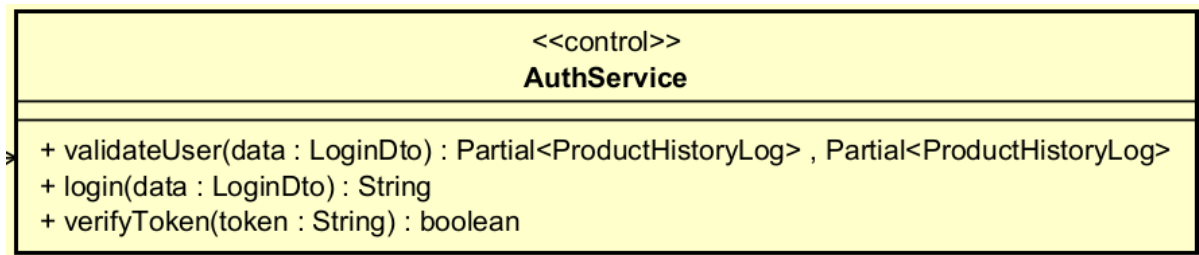


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	None			This service class does not define internal attributes in the current class diagram

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateUser(data : LoginDto)	User	Validate login credentials and identify the matched user
2	login(data : LoginDto)	String	Generate and return an authentication token or session identifier

3	verifyToken(token : String)	boolean	Verify whether the provided token is valid
---	-----------------------------	---------	--

Detailed specification of main operations

1. validateUser(data : LoginDto)

Parameter:

- data: login credentials used for authentication

Exception:

- ValidationException if login data is invalid
- AuthException if the credentials do not match any user

Method

- Validate the provided login credentials
- Check the user data source for a matching account
- Return the corresponding User entity if authentication data is valid

Other diagrams

- Sequence diagram if the validation process is shown

2. login(data : LoginDto)

Parameter:

- data: login credentials of the user

Exception:

- AuthException if login fails

Method

- Call validateUser(data)
- Generate an authentication token or session identifier
- Return the generated token as String

Other diagrams

- Sequence diagram if the token generation flow is shown

3. verifyToken(token : String)

Parameter:

- token: authentication token to be verified

Exception:

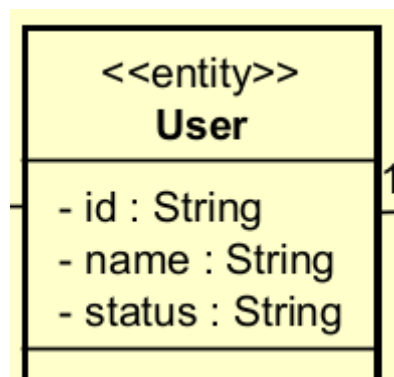
- AuthException if token validation encounters an error

Method

- Check the format and validity of the token
- Return true if the token is valid
- Return false otherwise

Other diagrams

- Activity diagram if the token verification logic is shown

53. Design for class “User”**Table 1. Example of attribute design**

#	Name	Data type	Default value	Description
1	id	String	generated by system	Unique identifier of the user
2	name	String	""	Name of the user

3	status	String	"Active"	Current status of the user
---	--------	--------	----------	----------------------------

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. Entity note

Parameter:

- None

Exception:

- None

Method

- User is an entity class representing a system user
- It stores the core identity and status information of the user
- One User may be associated with multiple ActionLogHistory records
- One User may be associated with one or more Role objects

Other diagrams

- Class diagram / ER-style relationship diagram if needed

54. Design for class “Role”

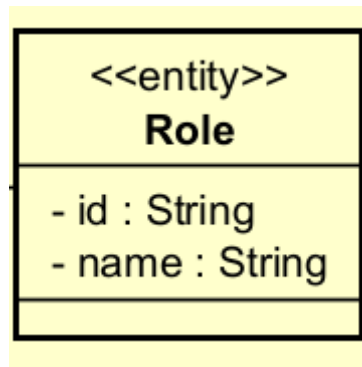


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	id	String	generated by system	Unique identifier of the role
2	name	String	""	Name of the role

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. Entity note

Parameter:

- None

Exception:

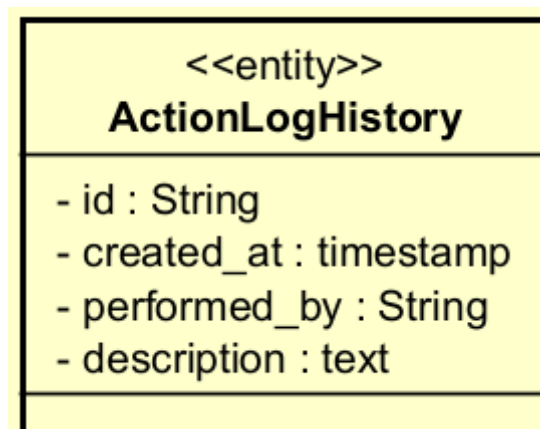
- None

Method

- Role is an entity class representing a role assigned to users
- It is used to define access level or system responsibility
- One Role may be assigned to one or more User objects

Other diagrams

- Class diagram / relationship diagram if needed

55. Design for class “ActionLogHistory”**Table 1. Example of attribute design**

#	Name	Data type	Default value	Description
1	id	String	generated by system	Unique identifier of the action log entry
2	created_at	timestamp	current timestamp	Time when the action log entry was created

3	performed_by	String	“”	Identifier or name of the user who performed the action
4	description	text	“”	Detailed description of the action

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. Entity note

Parameter:

- None

Exception:

- None

Method

- ActionLogHistory is an entity class used to store system or administrator activity logs
- Each log entry records who performed the action, when it occurred, and what happened
- Many ActionLogHistory records may belong to one User

Other diagrams

- Class diagram / relationship diagram if needed

56. Design for class “UserDto”

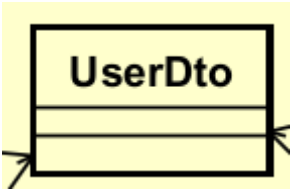


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	Not explicitly defined			The current class diagram does not specify the internal attributes of UserDto

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. DTO note

Parameter:

- None

Exception:

- None

Method

- UserDto is a data transfer object used to transfer user-related data between layers
- It is commonly used between frontend, controller, and service classes
- The detailed fields are not specified in the current class diagram

Other diagrams

- Class diagram if DTO structure is later extended

57. Design for class “ActionLogDto”

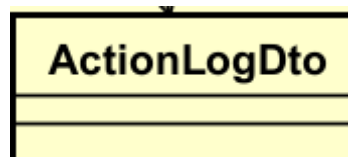


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	Not explicitly defined			The current class diagram does not specify the internal attributes of ActionLogDto

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. DTO note

Parameter:

- None

Exception:

- None

Method

- ActionLogDto is a data transfer object used to transfer action log data between layers
- It is typically used when returning log information from service/controller to the frontend
- The detailed fields are not specified in the current class diagram

Other diagrams

- Class diagram if DTO structure is later extended

58. Design for class “UserException”

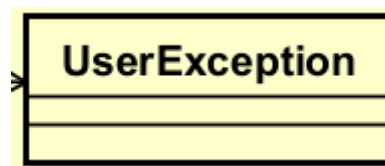


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	Not explicitly defined			The current class diagram does not specify the internal attributes of UserException

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	N/A	N/A	No public operations are explicitly defined in the current class diagram

Detailed specification of main operations

1. Exception note

Parameter:

- None

Exception:

- None

Method

- UserException is a custom exception class used to represent user-related business or processing errors
- It may be thrown by UserService or other related backend classes
- It helps standardize user-related error handling in the system

Other diagrams

- Sequence diagram if exception propagation is shown

59. Design for class "OrderResultPage"

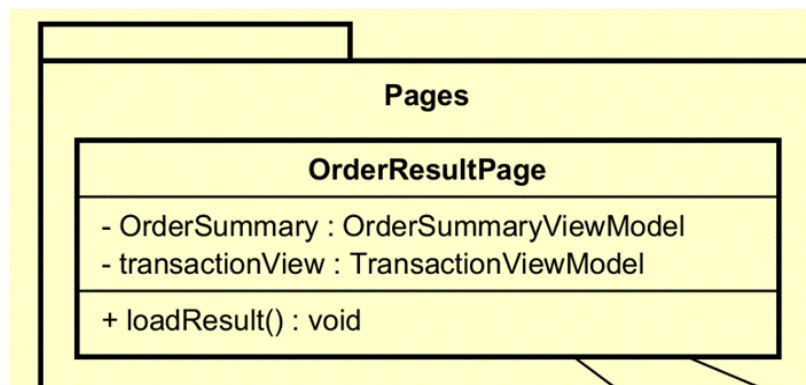


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	OrderSummary	OrderSummaryViewModel	null	Order summary
2	transactionView	TransactionViewModel	null	Transaction info

Table 2. Operation design

#	Name	Return type	Description
1	loadResult	void	Load order result

Parameter:

- None

Exception:

- None

Method

- Load the final order result after successful checkout
- Retrieve order summary data from the frontend API or service layer
- Retrieve transaction data from the frontend API or service layer
- Bind the returned OrderSummaryViewModel and TransactionViewModel to the page
- Display order ID, order status, receiver name, phone number, province, address, and total payment
- Display transaction ID, transaction content, transaction time, and amount
- Present the final confirmation information of the Place Order flow
- Do not perform payment processing, invoice generation, or delivery validation

Other diagrams

- None

60. Design for class "DeliveryInfoFormComponent"

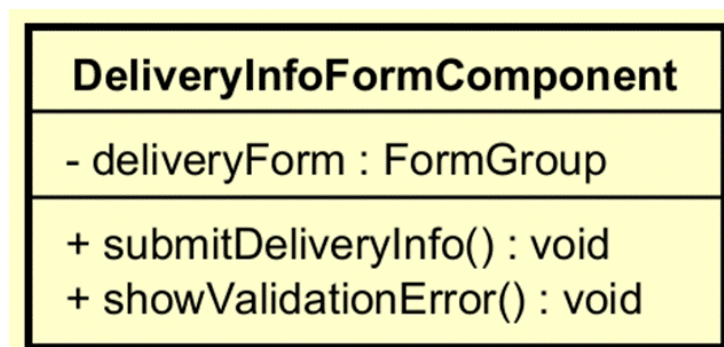


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	deliveryForm	FormGroup	null	Form data

Table 2. Operation design

#	Name	Return type	Description
1	submitDeliveryInfo	void	Submit delivery info
2	showValidationError	void	Show validation error

Parameter:

- None

Exception:

- Validation exception if one or more required delivery information fields are empty or invalid

Method

- Display the delivery information form to the customer

- Collect receiver name, email, phone number, province, address, and delivery notes from the input fields
- Create a delivery information data object from the entered values
- Submit the entered delivery information to the frontend API or service layer
- If any required field is missing or invalid, display validation error messages
- Request the customer to correct the invalid or incomplete information
- Allow the customer to resubmit the form after correction
- Do not calculate shipping fee, create invoice, or process payment directly

Other diagrams

- None

61. Design for class "OrderAPIService"

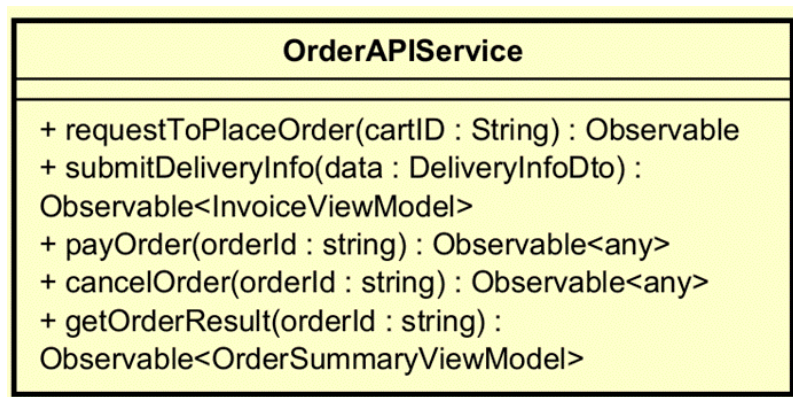


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description
1	requestToPlaceOrder	Observable	Request place order

2	submitDeliveryInfo	Observable	Submit delivery
3	payOrder	Observable	Pay order
4	cancelOrder	Observable	Cancel order
5	getOrderResult	Observable	Get result

Parameter:

- cartId for requestToPlaceOrder()
- data: DeliveryInfoDto for submitDeliveryInfo()
- orderId for payOrder()
- orderId for cancelOrder()
- orderId for getOrderResult()

Exception:

- HTTP exception if the backend endpoint is unavailable
- Network exception if the request cannot be completed
- Request exception if the submitted data is invalid or incomplete

Method

- Send the initial place-order request from the frontend to the backend
- Submit delivery information entered by the customer to the backend
- Receive invoice data returned from the backend after successful delivery information validation
- Send the payment request for the current order to the backend
- Send the cancellation request for the current order to the backend
- Retrieve the final order result data after successful checkout
- Return the backend response to the corresponding frontend page or component

- Do not perform business logic such as delivery validation, shipping fee calculation, invoice creation, or payment processing directly

Other diagrams

- Sequence diagram of Place Order can be reused

62. Design for class "TransactionViewModel"

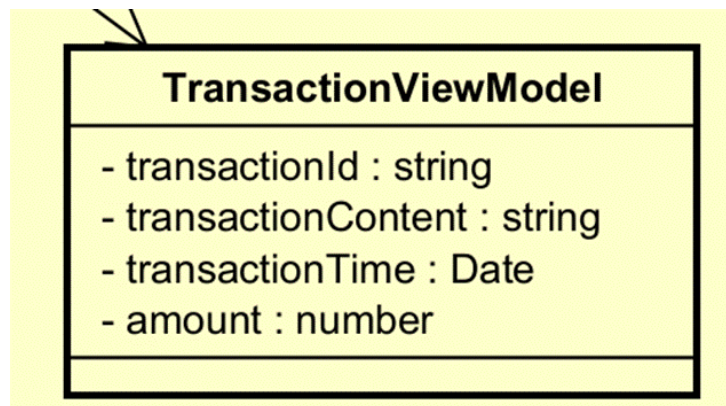


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	null	Transaction ID
2	amount	number	0	Amount
3	transactionContent	string	null	Description/content of the transaction.
4	transactionTime	Date	Now()	The time when the transaction occurred.

Table 2. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

Parameter:

- None

Exception:

- None

Method

- Store transaction information returned from the backend after successful payment
- Hold transaction ID for display on the result page
- Hold transaction content for display on the result page
- Hold transaction time for display on the result page
- Hold paid amount for display on the result page
- Provide transaction data to OrderResultPage for presentation
- Do not perform payment processing, validation, or transaction creation

Other diagrams

- None

63. Design for class "TransactionViewModel"

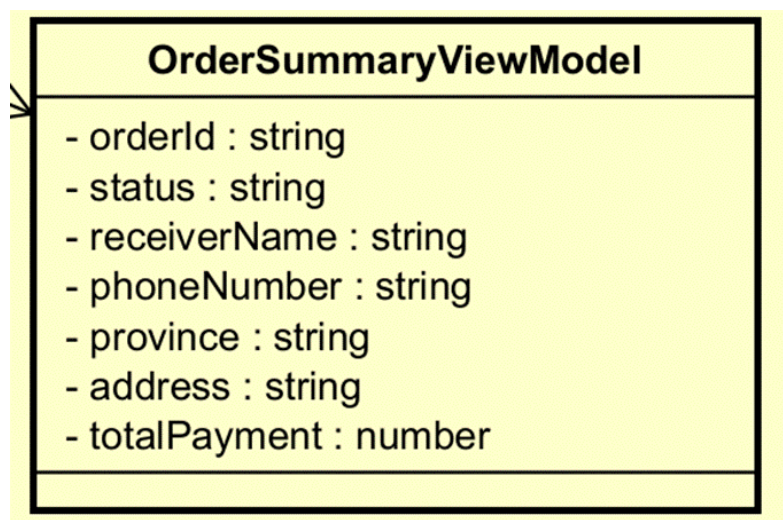


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	string	null	Unique identifier of the order
2	status	string	null	Current status of the order
3	receiverName	string	null	Full name of the receiver
4	phoneNumber	string	null	Phone number of the receiver
5	province	string	null	Province or city for delivery
6	address	string	null	Detailed delivery address
7	totalPayment	string	0	Total amount paid for the order

Table 2. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

Parameter:

- None

Exception:

- None

Method

- Store transaction information returned from the backend after successful payment
- Hold transactionId for display on the result page
- Hold transactionContent for display on the result page
- Hold transactionTime for display on the result page
- Hold amount for display on the result page
- Provide transaction data to OrderResultPage for presentation
- Do not perform payment processing, validation, or transaction creation

Other diagrams

- None

64. Design for class "DeliveryInfoDto"

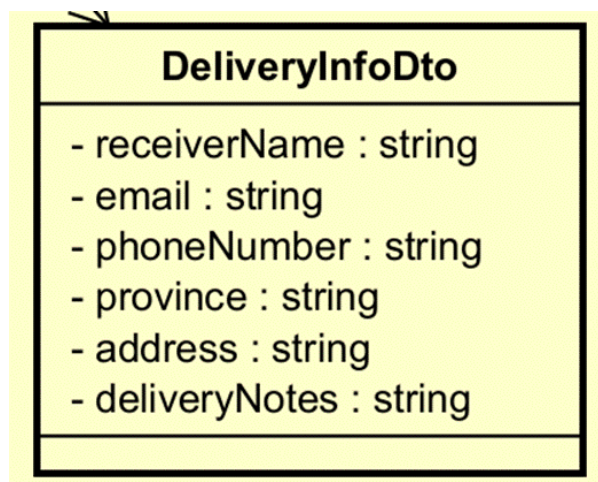


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	receiverName	string	null	Full name of the receiver transferred between layers

2	email	string	null	Email address used for sending invoice and transaction information
3	phoneNumber	string	null	Phone number used for delivery contact
4	province	string	null	Province or city for delivery location
5	address	string	null	Detailed delivery address
6	deliveryNotes	string	null	Optional notes for delivery instructions

Table 2. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

Parameter:

- None

Exception:

- None

Method

- Store delivery information entered by the customer on the frontend
- Hold receiverName for delivery identification
- Hold email for sending invoice and transaction information
- Hold phoneNumber for delivery contact
- Hold province for delivery location

- Hold address for detailed delivery address
- Hold deliveryNotes for optional delivery instructions
- Transfer delivery information from DeliveryInfoFormComponent to OrderAPIService
- Do not perform validation, shipping fee calculation, invoice creation, or payment processing

Other diagrams

- None

65. Design for class "OrderController"

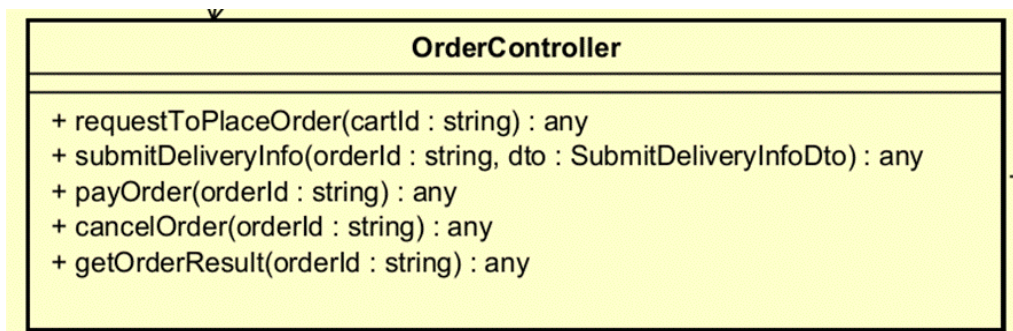


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description
1	requestToPlaceOrder	any	Request order
2	submitDeliveryInfo	any	Submit delivery
3	payOrder	any	Pay order
4	cancelOrder	any	Cancel order

5	getOrderResult	any	Get result
---	----------------	-----	------------

Parameter:

- cartId for requestToPlaceOrder()
- orderId and dto: SubmitDeliveryInfoDto for submitDeliveryInfo()
- orderId for payOrder()
- orderId for cancelOrder()
- orderId for getOrderResult()

Exception:

- Request exception if required request parameters are missing
- Validation exception if submitted delivery information is invalid or incomplete
- Service exception if the backend service cannot complete the requested operation

Method

- Receive the initial place-order request from the frontend
- Forward the checkout request to OrderService
- Receive submitted delivery information from the frontend
- Forward the submitted delivery information to OrderService for validation and invoice creation
- Receive the payment request for the current order
- Forward the payment request to OrderService
- Receive the cancellation request for the current order
- Forward the cancellation request to OrderService
- Receive the request for final order result information
- Return the processed result from OrderService to the frontend
- Do not implement business logic such as shipping fee calculation, invoice generation, payment processing, or email sending directly

Other diagrams

- None

66. Design for class "SubmitDeliveryInfo"

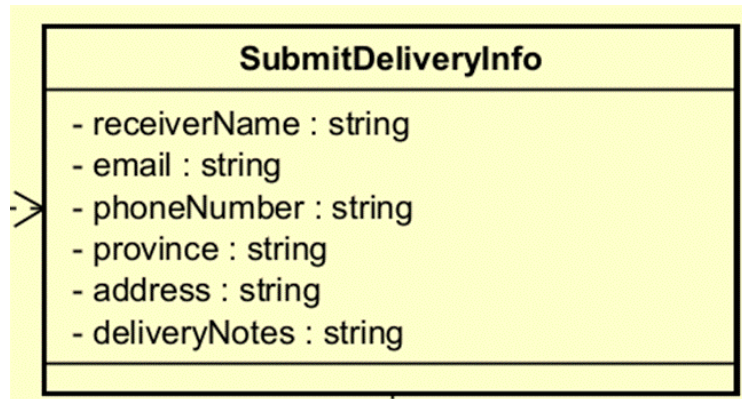


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	receiverName	string	null	Full name of the receiver submitted for delivery
2	email	string	null	Email address submitted for sending invoice and transaction information
3	phoneNumber	string	null	Phone number submitted for delivery contact
4	province	string	null	Province or city submitted for delivery location

5	address	string	null	Detailed delivery address submitted by the customer
6	deliveryNotes	string	null	Optional delivery notes submitted with the order

Table 2. Operation design

#	Name	Return type	Description

Parameter:

- None

Exception:

- None

Method

- Store delivery information submitted from the frontend to the backend
- Hold receiverName for identifying the order receiver
- Hold email for sending invoice and transaction information
- Hold phoneNumber for delivery contact
- Hold province for delivery location
- Hold address for detailed delivery address
- Hold deliveryNotes for optional delivery instructions
- Transfer delivery information from OrderController to OrderService

- Do not perform validation, shipping fee calculation, invoice creation, or payment processing

Other diagrams

- None

67. Design for class "OrderService"

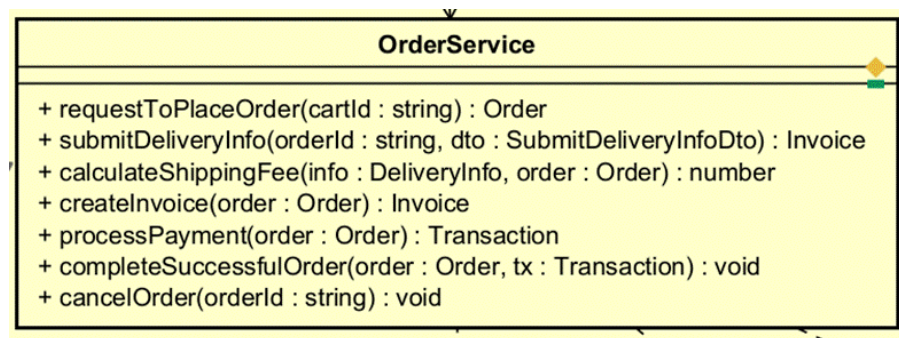


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description
1	requestToPlaceOrder	Order	Init order
2	submitDeliveryInfo	Invoice	Handle delivery
3	processPayment	Transaction	Process payment
4	cancelOrder	void	Cancel order
5	calculateShippingFee	number	Calculate shipping fee based on delivery information and order data

6	createInvoice	Invoice	Create invoice information for the current order
7	completeSuccessfulOrder	void	Complete the successful order flow after payment succeeds

Parameter:

- cartId for requestToPlaceOrder()
- orderId and dto: SubmitDeliveryInfoDto for submitDeliveryInfo()
- info: DeliveryInfo and order: Order for calculateShippingFee()
- order: Order for createInvoice()
- order: Order for processPayment()
- order: Order and tx: Transaction for completeSuccessfulOrder()
- orderId for cancelOrder()

Exception:

- Out-of-stock exception if one or more items in the cart do not have sufficient quantity
- Validation exception if submitted delivery information is incomplete or invalid
- Payment exception if payment fails or is canceled
- Persistence exception if order or transaction information cannot be recorded
- Notification exception if invoice and transaction information cannot be sent by email

Method

- Receive the place-order request from OrderController
- Check whether the cart contains at least one valid item

- Check whether the available stock is sufficient for all requested items
- Create a new Order when checkout can proceed
- Receive submitted delivery information from OrderController
- Convert the submitted DTO into DeliveryInfo
- Validate the delivery information before continuing checkout
- Calculate shipping fee based on delivery information and current order data
- Save delivery information into the current Order
- Create an Invoice for the order after successful delivery information validation
- Return invoice data to the frontend for display before payment
- Process payment for the current order
- Create a Transaction after successful payment
- Complete the successful order by clearing the cart
- Set the order status to PendingProcessing after successful payment
- Record order information in the system
- Record transaction information in the system
- Call EmailService to send invoice and transaction information to the customer's email
- Cancel the current order when cancellation is requested
- Do not render UI directly

Other diagrams

- None

68. Design for class "EmailService"

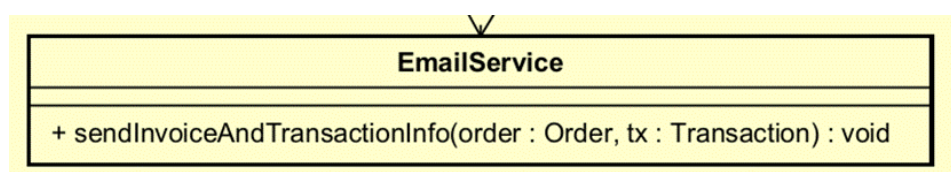


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description
1	sendInvoiceAndTransactionInfo	void	Send email

Parameter:

- order: Order and tx: Transaction for sendInvoiceAndTransactionInfo()

Exception:

- Email exception if the customer email address is missing or invalid
- Delivery exception if the email cannot be sent successfully
- Content generation exception if invoice or transaction information is incomplete

Method

- Receive the completed Order and successful Transaction from OrderService
- Extract the customer email from the delivery information of the order
- Prepare email content containing invoice and transaction information
- Include order summary information in the email
- Include transaction ID, transaction content, transaction time, and amount in the email
- Send the email to the customer after successful payment
- Support the post-checkout notification step of the Place Order flow
- Do not validate delivery information, process payment, or create invoice data directly

Other diagrams

- None

69. Design for class "Cart"

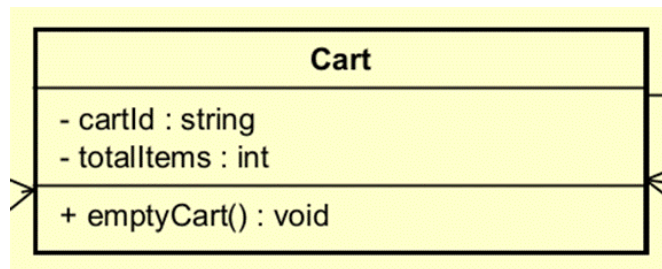


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cartId	string	null	Cart ID
2	totalItems	int	0	Total items

Table 2. Operation design

#	Name	Return type	Description
1	emptyCart	void	Clear cart

Parameter:

- None

Exception:

- Cart exception if the cart does not exist
- Cart state exception if the cart is empty when checkout is requested

Method

- Store the items selected by the customer before checkout
- Hold the current number of items in the cart

- Support the Place Order flow as the source of checkout data
- Allow the system to check whether the cart contains at least one valid item
- Be cleared after successful payment by calling emptyCart()
- Preserve cart information when the ordering process is canceled or paused
- Do not validate delivery information, process payment, or create invoice data directly

Other diagrams

Sequence diagram of Place Order can be reused

70. Design for class "DeliveryInfo"

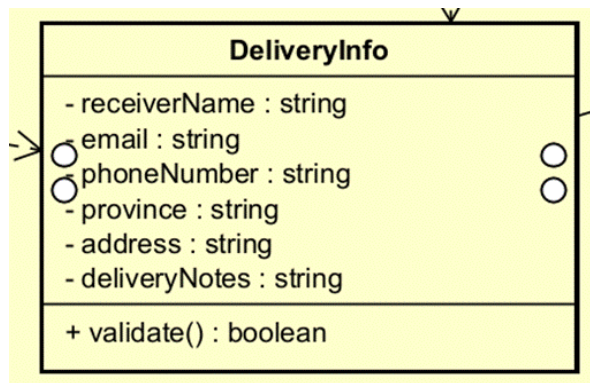


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	receiverName	string	null	Full name of the receiver of the order
2	email	string	null	Email address used to receive invoice and

				transaction information
3	phoneNumber	string	null	Phone number of the receiver for delivery contact
4	province	string	null	Province or city where the order will be delivered
5	address	string	null	Detailed delivery address
6	deliveryNotes	string	null	Additional notes or instructions for delivery

Table 2. Operation design

#	Name	Return type	Description
1	validate	boolean	Validate info

Parameter:

- None

Exception:

- Cart exception if the cart does not exist
- Cart state exception if the cart is empty when checkout is requested

Method

- Store the items selected by the customer before checkout

- Hold the current number of items in the cart
- Support the Place Order flow as the source of checkout data
- Allow the system to check whether the cart contains at least one valid item
- Be cleared after successful payment by calling emptyCart()
- Preserve cart information when the ordering process is canceled or paused
- Do not validate delivery information, process payment, or create invoice data directly

Other diagrams

- Sequence diagram of Place Order can be reused

71. Design for class "Order"

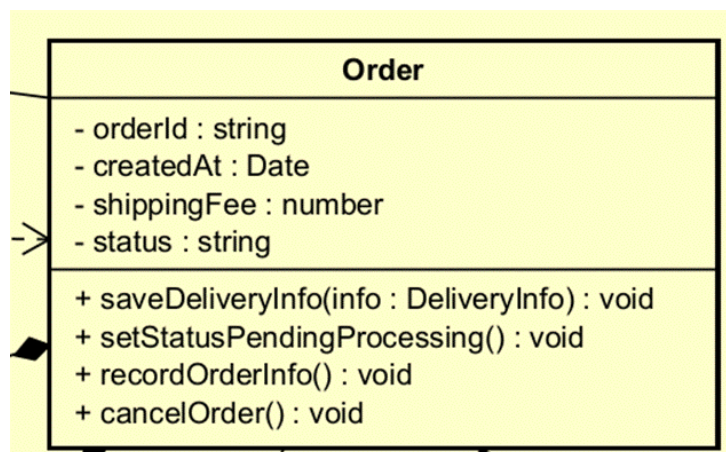


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	string	null	Order ID
2	createdAt	Date	now()	Date and time when the order is created

3	shippingFee	number	0	Shipping fee calculated for the order
4	Status	string	Draft	Current processing status of the order

Table 2. Operation design

#	Name	Return type	Description
1	setStatusPendingProcessing	void	Set status
2	cancelOrder	void	Cancel order
3	recordOrderInfo	void	Record successful order information into the system
4	cancelOrder	void	Cancel the current order

Parameter:

- info: DeliveryInfo for saveDeliveryInfo()

Exception:

- State exception if the order cannot transition to the required status
- Persistence exception if order information cannot be recorded successfully
- Validation exception if the provided delivery information is missing or invalid
- Cancellation exception if the order cannot be canceled in its current state

Method

- Store the core information of the current order
- Hold orderId as the unique identifier of the order
- Hold createdAt as the creation time of the order
- Hold shippingFee as the calculated shipping fee of the order
- Hold status as the current processing state of the order
- Receive and save validated delivery information through saveDeliveryInfo()
- Update the order status to PendingProcessing after successful payment through setStatusPendingProcessing()
- Record successful order information in the system through recordOrderInfo()
- Cancel the current order through cancelOrder() when cancellation is requested
- Support the Place Order flow as the main business entity created during checkout
- Do not validate delivery information, process payment, or send email directly

Other diagrams

- State machine diagram of Order can be reused

72. Design for class "Invoice"

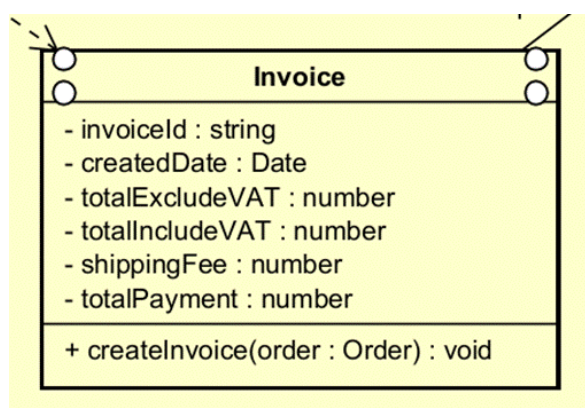


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	invoiceId	string	null	Invoice ID
2	createdDate	Date	now()	Date and time when the invoice is created
3	totalExcludeVAT	number	0	Total amount of all ordered items before VAT
4	totalIncludeVAT	number	0	Total amount of all ordered items including VAT
5	shippingFee	number	0	Shipping fee calculated for the order
6	totalPayment	number	0	Final amount to be paid including VAT and shipping fee

Table 2. Operation design

#	Name	Return type	Description
1	createInvoice	void	Create invoice

Parameter:

- order: Order for createInvoice()

Exception:

- Invoice generation exception if the order data is incomplete
- Calculation exception if invoice totals cannot be computed correctly
- Data exception if required order information is missing

Method

- Store invoice information associated with the current order
- Hold invoiceId as the unique identifier of the invoice
- Hold createdDate as the time when the invoice is generated
- Hold totalExcludeVAT as the total product amount before VAT
- Hold totalIncludeVAT as the total product amount after VAT
- Hold shippingFee as the shipping cost of the order
- Hold totalPayment as the final amount to be paid by the customer
- Generate invoice data from the current order through createInvoice()
- Calculate invoice totals after delivery information is validated and shipping fee is determined
- Provide invoice information for display on the invoice page before payment
- Do not validate delivery information, process payment, or send email directly

Other diagrams

- None

73. Design for class "Transaction"

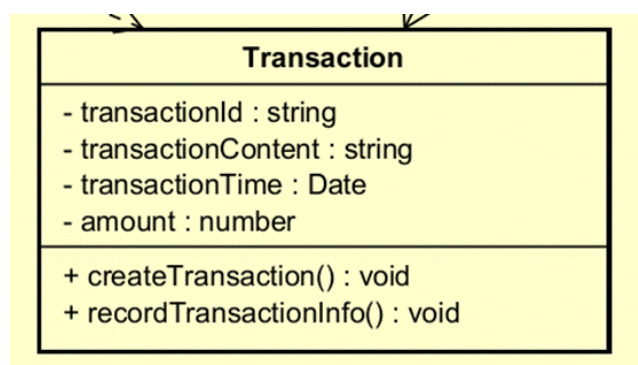


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	transactionId	string	Null	Transaction ID
2	transactionContent	string	Null	Content or description of the payment transaction
3	transactionTime	Date	now()	Date and time when the transaction occurs
4	Amount	Number	0	Amount paid in the transaction

Table 2. Operation design

#	Name	Return type	Description
1	createTransaction	void	Create transaction
2	recordTransactionInfo	void	Record transaction information into the system

Parameter:

- None

Exception:

- Transaction creation exception if payment result data is incomplete
- Persistence exception if transaction information cannot be recorded successfully
- Payment exception if the payment process fails before transaction creation is completed

Method

- Store transaction information associated with the current order
- Hold transactionId as the unique identifier of the transaction
- Hold transactionContent as the payment description or transaction message
- Hold transactionTime as the time when the transaction occurs
- Hold amount as the paid amount of the transaction
- Create transaction data after successful payment through createTransaction()
- Record transaction information in the system through recordTransactionInfo()
- Provide transaction information for display on the order result page after successful checkout
- Support the Place Order flow as the payment result entity
- Do not validate delivery information, create invoice data, or send email directly

Other diagrams

- None

74. Design for class "OrderItem"

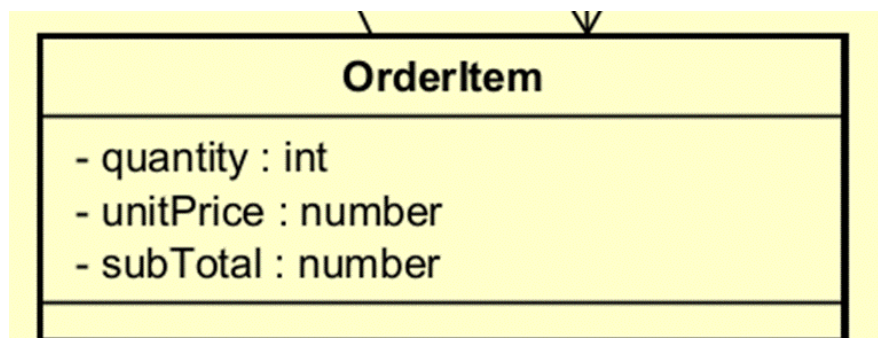


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	Quantity	int	1	Quantity

2	unitPrice	number	0	Price of one unit of the ordered item
3	subTotal	number	0	Calculated total amount for this order item

Table 2. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

Parameter:

- None

Exception:

- Quantity exception if the item quantity is less than or equal to zero
- Price exception if the unit price is invalid or negative
- Calculation exception if the subtotal cannot be computed correctly

Method

- Store a single line item of the current order
- Hold quantity as the number of units ordered
- Hold unitPrice as the price of one unit of the item
- Hold subTotal as the calculated amount for the line item
- Support invoice generation by providing item quantity and price information
- Support order summary display by representing each purchased item in the order
- Be associated with the current order as one of its ordered items
- Do not validate delivery information, process payment, or send email directly

Other diagrams

- None